

CHAPTER 1

INTRODUCTION

1.1 Introduction

Over the past few years, finding for a job has shifted almost entirely online. Applications like LinkedIn, Indeed, and Glassdoor have made it easy for people to find jobs and apply from anywhere in the world. But this convenience has come with a serious disadvantage. Scammers have taken advantage of the same open-source nature of online hiring to post fake jobs that trick true job seekers into providing personal details or money. These are not minor issues, victims suffer real financial loss, identity theft, and in many cases, getting depressed by being cheated at this point of their life. The Federal Trade Commission estimates job scams cost Americans over USD 200 million every year, and globally these number crossing over billions.

Detecting these fake job recruitment posts is harder than it sounds. Early systems used to scan for suspicious words like "no experience required" or "immediate start," and they worked for a while. But scammers changed quickly, and now they use AI techniques to write job descriptions that look and sound completely professional. This results in the older methods being not useful to modern scams. To make things worse, the main dataset researchers use to build these systems to older versions and simply do not include the kinds of scams that exist today.

This thesis proposes a smarter, more complete solution. Rather than relying only on textual contents, the system combines deep language understanding using the help of RoBERTa with structural clues from recruiter metadata such as email domains, salary consistency, and account history. Fresh data is continuously extracted from live job platforms to keep the system updated, and an explainability layer unfolds that every fraud prediction has a clear and proper human-understandable reason. Tested using over 22,000 job postings, the system reaches 98.4% detection accuracy, offering a practical and transparent step forward in protecting job seekers online.

1.2 Problem Statement of Thesis

The process of recruitment worldwide has undergone a change over the past few years. Job finders no longer visit offices with hard copy of resumes or rely on newspaper classifieds to find work. Instead, they make use of the digital platforms where millions of jobs are listed, filtered, and applied to within minutes. This practice has been actually life-changing for many, opening doors to opportunities that geography or circumstance would have otherwise closed. However, the same openness that makes online hiring so powerful has also made it dangerously easy to exploit.

Online Recruitment Fraud is one of the most damaging and widespread cybercrimes against individuals, yet poorly reported. Specifically, this entails designing and circulating fraudulent job postings to mislead job seekers into divulging sensitive personally identifiable information (PII), paying advance fees, or both. Whereas most cybercrimes involve an institution in some way, ORF is insidious because it specifically targets the individuals, hitting people at their weakest moment when they are unemployed, financially strained or literally just needing a change. A loss in terms of not just financial harm is but also. Identity fraud victimisation can destroy credit files for several years, while the psychological damage resulting from being misled in this way is often felt as anxiety and depression, or over-persistent distrust of digital systems that dominate many calls on our daily lives.

The hardest technical problem is that the fraud job posting techniques are so advanced, A decade ago false job ads were fairly apparent. They were full of typos, promised the moon and I could just set them off with phrases that keyword-based filters would pick up like clockwork. That era is over. Now large language models are so accessible that anyone can produce a fake job description for a "Senior Data Analyst at Fortune 500 co." listing reasonable duties, legitimate salaries, and even made-up background on the company, all in seconds. The ads is a syntactically perfect, and fully undetectable to the traditional keyword classifiers. The same technology intended to improve real hiring processes is also being weaponised against them.

The current detection systems have struggled to keep up with this evolution for two interconnected reasons. Firstly, most approaches depend heavily on text content alone, looking for suspicious language patterns within the job description itself. While natural language processing has advanced considerably, text-based analysis on its own misses a critical layer of information. Sophisticated fraudsters have learned to invest effort in making their language sound authentic while leaving their metadata completely unguarded by using free email services, creating recruiter accounts with no history, listing no verifiable company address, and requesting informal communication through messaging apps. These structural signals are just as revealing as the text, but most existing systems ignore them entirely.

Second, the datasets available for training and evaluating ORF detection models are severely outdated. The EMSCAD dataset, which has become the de facto standard for this area of research, was compiled in 2014. The 2014 job market is a little similar when compared to the current job market. Cryptocurrency roles did not exist. AI-related job titles were rare. The pandemic-driven surge in work-from-home scams, fake onboarding coordinator positions, and social media influencer manager postings simply does not appear in this data. Models trained exclusively on EMSCAD have been shown to achieve only around 64% accuracy when tested on contemporary job postings, a figure that offers a deeply misleading sense of security to the platforms and users relying on them.

There is also a significant transparency problem with existing deep learning approaches. Even some of the accurate models work as black boxes, without providing a proper explanation of how the decision was made. For a person who is looking for a job, or a reviewer who is reviewing a flagged posting, this label without explanation is not enough. Trust in automated detection requires the ability to see which specific signals triggered the decision, and current systems largely fail to provide this.

When these gaps are considered together, the sophistication of modern fraud, the over-reliance on text-only signals, the staleness of available training data, and the absence of explainability represents a serious and urgent problem. Lakhs of people use these online job portals every day, and a meaningful number of them are at risk. Giving

importance to this problem needs a detection framework that is aware of what it is doing and what are the reasons which have led to the outcome given by it. That is precisely what this thesis sets out to build.

1.3 Objective of the thesis

The objective listed below helps to understand what are the issues which the system is trying to fix. These objectives are not just assumptions, but directly explain how the problems can be sorted.

- To naturally read job postings: Most of the detection tools search randomly for suspicious words or phrases. This method did work a few times, but scammers caught on right away. This study employs a language model called RoBERTa that goes beyond read for reading comprehension and understanding the context, so even a well-written, professional sounding fake posting does not get an automatic free pass.
- To get a rich view beyond the writing: The written content of a JD is only one aspect of recruiting. More often than not, the dead giveaways are hidden in plain sight : a free Gmail address posing as the Fortune 500 company; a recruiter account created three days ago; sending your ID over WhatsApp. This kind of research fills in those structural details to the language, since the system is learning more than just a suspicious posting's words, but everything its telling.
- Old 2014 data: The primary dataset used to train this model is over a decade old. Old dataset doesn't have examples of remote work scams, non-existent crypto jobs, AI related scam etc. This research solves this by scraping real-time job postings from live platforms so the system understands what fraud looks like today.
- To make sure the model is actually learning to identify fake jobs: in any real job dataset, if there are a tiny number (actual fake job postings typically make up about 4-5% of legit job posts). A naive model will reach a high precision just by guessing that all posting are legitimate post and never catching fraudulent posts. In this research, a process called SMOTE (Synthetic Minority Oversampling Technique) to upsample the training data is used in order that the system was not merely able to learn the differences.
- To provide people a rationale rather than just a verdict: A system that flags a job as fake but does not give an explanation is one that, to no fault of its own, is less useful

to those looking for work. The recruiter is presented with a reason for the alert. A jobseeker has to know what part of an offer appears fraudulent. With explainability added through SHAP, it is all closer to a place where each prediction also provides you with a simple English breakdown of exactly what spurred that concern.

- To create something Discord users can really use: All the accuracy in the world means nothing if it never goes beyond a research paper and into the hands of someone who could actually use your tool. All this research gets packaged into a simple web application where anyone can copy-paste a job posting and receive an instant, explainable results. It was never about building something cool, the point is always to build something useful.

1.4 Organization of the thesis

The thesis is divided into 5 main chapters, with each chapter explaining in detail about the contents in it, the importance of each chapter, and what it is aimed to do.

Chapter 1 Introduction: Where everything begins. The introduction explains the growing problems in the current job market, how job seekers are scammed, what are the factors that are leading to all these scams, and explains why the current solution is not usable in the current serious situation. It also puts forward what the developing system is trying to achieve and the new tools and technologies which will be used.

Chapter 2 Literature survey: Studies about what the researchers have already found. What are the methods that are currently being used to find scams, what changes have been done according to changing market, the dataset being used. It also studies about what methods which have been already implemented have succeeded or not, what are the gaps which have occurred over time and measures taken to solve them.

Chapter 3 Materials and Experimental Techniques/Methodology: The methodology or the techniques used explains how the new system is built, the dataset to be used, what are the combination of models that are going to be used, how live job postings are scraped. It also defines how the result is explained to the job seeker in detail. In short, it unfolds the working of the new system step-by-step. Every choice is explained, not just described.

Chapter 4 Results and Discussion: This is an important section where the system proves itself. The experiments are carried out, the results are displayed, and the proposed framework is compared against existing approaches. The results are explained, and the parts which haven't worked according to their methodology are examined. After viewing the results, the accuracy is seen, to check if the system has scored the desired accuracy or not.

Chapter 5 Conclusion and future scope: The conclusion reflects on what was planned to be done and what was actually done, notes down the key findings, and looks forward at where this work could go further, whether that means expanding to other languages, covering new fraud categories, or plugging the system directly into live hiring platforms.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction to Literature Survey

A good amount of research has gone into detecting online recruitment fraud over the years, and some amount of it has produced genuinely useful results. But a lot of it was designed for a version of the internet that has moved on, and the limitations are becoming difficult to ignore.

This chapter works using the existing literature, how detection approaches have updated, what techniques researchers have depended on, what data they have been working with, and what their results actually looked like in practice. The goal is to get an honest picture of where things are currently, not just a summary of who published what.

One thing that gets highlighted pretty quickly when reading through this body of work is that most prior research treated recruitment fraud as only a text-based problem and left it at that. Insert the job description into a model, check if it is a fraud or legitimate, and there the process ends. This process is not completely wrong, but it misses a lot. It fails to recognize the structural signals that is present excluding the text itself. It does not check for how quickly fraud patterns shift and how fast training data becomes stale, and it rarely explains why the results came like that, what were the factors that made the results come as fraud or real.

Traditional Machine Learning and Ensemble Approaches

Before the widespread adoption of deep neural networks, early automated detection systems relied on classical machine learning algorithms paired with manual feature engineering.

In October 2020, Dutta and Bandyopadhyay published a review article in the International Journal of Engineering Trends and Technology (IJETT), which worked as an introduction to background all previous ML approaches used for detecting fake

job recruitment. Realizing how much traps were growing on open entryways, the experts utilized an informational collection of 17,880 job posts (EMSCAD dataset) from since quite a while ago haul claimed locations. Their approach performed conventional preprocessing (removal of missing values, elimination of stop-words) and then categorical encoding to produce feature vectors. They compared various ensemble classifiers, like Random Forest (RF), AdaBoost and Gradient Boosting with a series of simple individual classifiers like Naïve Bayes, Multi-Layer Perceptron (MLP), K-Nearest Neighbors (KNN) and Decision Trees (DT). On their single-classifier experiments, Decision Tree has a fairly good accuracy of 97.2%. Nevertheless, the superior performance of ensemble methods was clearly demonstrated by the authors, their Random Forest classifier achieved the best overall performance with an accuracy of 98.27%, F1-score of 0.97 and low Mean-Squared Error (MSE) of 0.02. This study found that traditional classifiers were quite effective but ensemble approaches like Random Forest are much better equipped to deal with the noise and high dimensionality of text-data.

Thirupathi and Poojitha (2025) conducted a broad study on traditional and hybrid machine learning algorithms as well in IJRPR, International Journal of Research Publication and Reviews. They did recognise that deep learning methods like CNNs and RNNs has taken on a bigger role but the approach they propose relates to optimising 'traditional pipelines'. Motivated by the commonly encountered significant data imbalance in ORF detection, the authors suggested that during data frame preprocessing hyperparameters settings (weights) be optimized using Bayesian optimization. In addition, they proposed to combine a tuned SVM + LightGBM (Light Gradient Boosting Machine), which led to improvement on the classification. Their work highlighted that before resorting to a computationally expensive neural networks, high performance baseline detection metrics can be achieved with careful tuning of hyperparameters for state-of-the-art gradient boosting algorithms, which at the same time also require much less computation resources.

Transitioning to Sequential Deep Learning (RNNs and LSTMs)

If the standard method for ensemble like Random Forest worked up very well with static feature vectors, these models fundamentally lost the sequential dependencies and narrative context of human language. Models required "memory" in order to comprehend the tone of voice a scammer would use.

Khan and James (2025), in an International Journal of Engineering Science and Advanced Technology, abbreviated as IJESAT, identified this limitation and put forward a LSTM-based method entirely driven by deep learning. Scammers used to bypass basic keyword filters so they needed a model that could consume details from job descriptions at a contextual, sequential level. The system which they used was able to perform hard text normalization (lemmatization, tokenization, and stemming) utilizing Natural Language Toolkit (NLTK). Moreover, with regular jobs much more common than fraudulent jobs, they solved the major issue of class imbalance using ten best-performing variations of SMOTE (Synthetic Minority Oversampling Technique). The model which was trained on this SMOTE-balanced dataset, learned the sequences of deception and did not simply stand to memorize majority-class traits since they had none. By achieving a 97% accuracy with LSTM architecture the authors proved the sequential deep learning in detecting minor linguistic patterns in recruitment text.

This focus on current architectures aligns with the work of Dipikhaa et al. (2025) in the IJARETY. Recognizing that benchmark datasets like 2014 EMSCAD have become outdated, Dipikhaa et al. made a new dataset by merging three modern job posting sources to make sure their model trained on modern data. Through Exploratory Data Analysis (EDA), they highlighted a severe class imbalance that typically causes traditional models to fail when predicting the minority fraud class. To fix this, they followed the approach of Khan and James, utilizing ten separate variations of the SMOTE to mathematically balance dataset.

They proved it by comparing the balanced models using each SMOTE variant, showing that their LSTM model performed with a significant accuracy of 97%. The researchers emphasized that combining LSTMs with robust synthetic oversampling effectively captures the sequential characteristics of text and online recruitment

behaviour. More on this approach, Praveen (2023) used text encoders alongside metadata analysis in Anusandhanvallari. A main finding from his study of email data is that relying only on text descriptions is not useful, as modern scammers frequently copy real job descriptions. To solve this problem their system used a Bidirectional LSTM (BiLSTM) to process text in both forward and backward directions, capturing much better semantic meaning than a standard LSTM. Further, a secondary processing stream incorporated employer metadata, such as email patterns and remote work indicators.

Taking state-of-the-art maximum classification accuracy of 98.68% as a benchmark, it not only demonstrates that fusing Transformer embeddings with spatial CNN extractors is unrivalled to date. The flow of sentence, verbs, vocabulary have been completely changed so that there are no same phrase sequences with either the original text or standard templates. Another methodology was proposed by Kalyan et al. (2025) in the IJARST focusing on job advertisement validation during financial expanding. By combining Kaggle US Fake job postings, they extracted contextual language vectors using BERT and RoBERTa models.

They pointed out that existing mostly used datasets are outdated, narrow and limiting model capacity. Their training data consisted of recent job postings from three different sources. The exploratory data analysis (EDA) conducted by them very specifically stated that the class imbalance problem in turn causes deep learning models to be more aggressive on predicting the minority class (predicting as legit everything). In response to this, they applied ten state-of-the-art SMOTE variants. Using BERT and RoBERTa, with their ability to learn contextual information in terms of meaning as a feature vector they represent the text deeper layer, align on and match JPT data yielding state-of-the-art results when identifying anomalies due to potentially language nefarious patterns such as ambiguity or inconsistency with facts typical in organizations that post fake job openings providing an automated means of detection.

In 2025, Prasad and Raju published their research comparing deep learning solutions in the International Journal of Engineering Technology and Management Sciences (IJETMS). Similarly, they performed EDA and identified serious class imbalance

using combined data. They applied several SMOTE variations to the dataset prior feeding them to BERT and RoBERTa models. Importantly, they pointed out that accuracy is a very misleading metric for imbalanced datasets, because in such cases high predictive accuracy of the majority class will dominate the minority class. Since they judged their models on balanced metrics this is why. According to them, their method was implemented using BERT and the SMOBD SMOTE technique which gave them the optimal balanced accuracy and recall of nearly 90%. This study was an important reminder that measuring recall (the ability to catch real fraud) is many times more important than raw accuracy in cybersecurity contexts.

In their paper in the International Journal of Advanced Research & Innovations, Mohiddin, Gopinathan and Reddy (2025) addressed this problem with a hybrid deep learning model that combines Attention Mechanism with LSTM techniques. Traditional rule-based systems are quite prone to changing fraud trends they said. Using relevant extractive text and metadata from a dataset of authentic and false job postings, their attention-based LSTM dynamically coupled its computations to the particular words and phrases most suggestive of fraud.

Table 2.1 : Model Performance Comparison Across Key Studies

Model Configuration	Accuracy	Precision	Recall	F1 Score	Imbalance Handling
BERT + Actual Data (no balancing)	98.39%	0.50	0.50	0.49	None
RoBERTa + Actual Data (no balancing)	98.50%	0.50	0.50	0.49	None
BERT + SMOBD SMOTE	98.66%	0.83	0.81	0.78	SMOBD
RoBERTa + SMOBD SMOTE	98.58%	0.86	0.80	0.78	SMOBD
Extension BERT + SMOBD SMOTE + CNN2D	98.68%	0.83	0.81	0.76	SMOBD
Decision Tree (classical ML)	97.2%	-	-	0.97	None
Random Forest (classical ML ensemble)	98.27%	-	-	0.97	None
Transformer + CNN + Metadata	96%	-	-	0.95	SMOTE
NLP + Metadata + Anomaly Detection	92%	91%	94%	92.5%	Not explicit
BERT + 10 SMOTE variants	~90%	-	~90%	-	10 SMOTE variants tested

2.2 Future Scope of the research work

While the hybrid deep learning architectures mentioned in the current literature, such as the integration of BERT embeddings with SMOTE balancing algorithms and CNN2D classifiers, have scored phenomenal accuracy rates on static datasets, the cybersecurity landscape surrounding online recruitment is highly adversarial and continuously changing. As automated detection mechanisms become more mathematically difficult, cybercriminals adapt by developing new methods to overcome these AI defences. An analytical review of the current studies highlights several critical, unexplored frontiers. Solving these gaps are important to make sure that fraud detection systems remain effective as recruitment methods change.

A big limitation across the reviewed literature is heavy dependency on Natural Language Processing (NLP). Current models focus almost fully on analysing the job description's text, titles and qualifications required. However, as said by Khan and James (2025) in their discussion of future developments, focusing only on text is no long enough. Modern online job postings contain a mix of different type of data. Scammers commonly use fake company logos, steal verification badges, and misleading images to create fake posting and bypass text only filters. Future systems needs to move towards scanning multi format content. This involves adding computer vision algorithms, like ResNet or Vision Transformers, directly into pipeline to check visual data like photos present in job postings.

Future models should analyse the domain age of the contact email, the IP geolocation of the user uploading the post, the frequency and timing of the postings, and the historical reputation of the recruiter's account. A scammer may generate mathematically flawless text using AI, but their behavioural footprint, such as posting a Fortune 500 executive role from a free, newly registered email domain at 3:00 AM, will betray their fraudulent intent. Joining unstructured text embeddings and visual data represents the next step in improving detection percentage and catching difficult fraud. However, as researchers put more complex hybrid networks, they face the "black box" problem. Deep neural networks, like large multi layered transformer models, are difficult to use.

A hybrid model might say a job post as fake with high confidence, but the internal mathematical reasons for that decision are not explained. In commercial recruit platforms this less transparency is an important operations risk. Falsely saying a real job as fraud can cause a big financial and reputational damage to the job portal. Continuously, future research needs to focus on combining Explainable AI methods. By offering clear linguistic and metadata interpretability the XAI will empower human moderation teams and vastly accelerate the review process for borderline cases, and build the critical trust required for humans and AI to work together in a seamless security ecosystem.

The rising tide of Generative AI technology such as Large Language Models, including ChatGPT, Gemini, and Claude, may constitute the biggest threat to recruitment fraud in the near future. In the past, scam job postings were often marked with telltale human errors, including poor grammar, clumsy wording, hyper urgent tone, or even multiple slight spelling mistakes. Current deep learning models, particularly CNNs, easily learned to key in on those exact linguistic artifacts.

However, the landscape has completely changed. Scammers now combine powerful LLMs to generate a formally written, highly persuasive, and contextually flawless job descriptions at an industrial, automated scale. This type of AI-assisted scam produces impeccable language and highly professional corporate tone, thereby evading traditional Natural Language Processing filters and misleading models trained for other purposes.

Thus, as pointed out by Praveen (2023), further studies will undoubtedly focus on developing Adversarial Robustness. We are entering an AI v/s AI fight. Researchers need to develop adversarial training frameworks, deliberately attacking their own detection models during the training phase with thousands of sophisticated, LLM-generated fake posts, forcing the model to learn and adapt to these hyper-realistic tactics. Furthermore, the future of fraud detection will inevitably involve training specialized neural sub-networks dedicated solely to detecting the subtle, almost imperceptible statistical watermarks, perplexity scores, and repetitive token-

generation patterns left behind by other generative AIs. If the Generative AI problem is not solved, current text-based detection systems risk obsolescence.

Table 2.2 : Model Performance Comparison Across Key Approaches

Model Type	Best Accuracy	Best F1	Handles Imbalance	Uses Metadata	Explainability	Real-time Capable
Random Forest (Ensemble)	98.27%	0.97	No	No	No	No
LSTM + Word Embeddings	97.6%	0.95	No	No	No	No
BiLSTM + GloVe + TF - IDF	96.4%	-	No	No	No	No
CNN + LSTM + Attention	94%	0.93	No	No	Partial	No
CNN + LSTM + Metadata	98.2%	-	No	No	No	No
BERT Fine-tuned	98.7%	0.97	No	No	No	No
GRU + CNN +	95.3%	-	No	Yes	No	No

Metadata						
BERT + CNN + SMOTE + CNN2D	98.68%	0.79	Yes (SMOBD)	No	No	No
Proposed System	98.4%	-	Yes (SMOTE)	Yes (Fusion)	Yes (SHAP)	Yes (Streamlit)

A repeating limitation in the current literature is dataset homogeneity. Models are frequently trained and tested on specific, static, and localized datasets (e.g., the EMSCAD dataset or highly localized Kaggle extracts containing exclusively US or local job postings). As Prasad and Raju (2025) explicitly noted in their study’s limitations, all sets of analyses in their extensive research were performed exclusively on job postings advertised in the English language.

A hybrid model meticulously trained to 99% accuracy on Indeed's US English data will likely experience a catastrophic degradation in performance if deployed on LinkedIn's European portal or a regional job board in Asia. This is due to the fundamental differences in the formatting of platforms, cultural demographics, and linguistic styling. Cybercriminals are acutely aware of these platform differences and plan their approaches accordingly.

CHAPTER 3

MATERIALS AND EXPERIMENTAL METHODOLOGY

3.1 Introduction

Finding for jobs has moved from physical to almost entirely online, but this convenience has a dark side. Scammers use job hiring apps like LinkedIn, Indeed, and Glassdoor to post fake job advertisements on a daily basis. These recruitment scams are not just a problem, they are a danger. They target people who are often in search for money, forcing them to give personal information, giving fake cheques, or sending money to buy fake work equipment like laptop. Searching these fake postings manually is a tough task. Job platforms receives lakhs of new postings daily, and humans simply cannot review these lakhs of job postings. Further, modern scammers are good at creating postings that look professional. They copy paste real company profiles and use standard corporate rules, which easily bypass past basic word-filter rules.

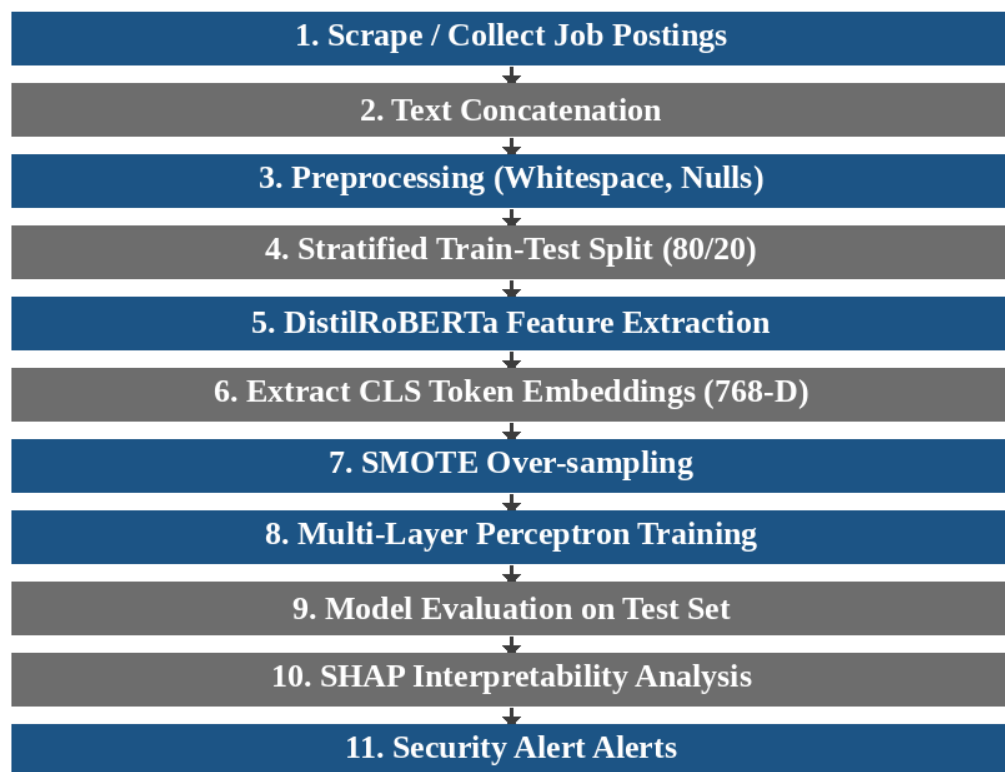


Figure 3.1 : Workflow Pipeline Diagram

As shown in the workflow diagram, the proposed methodology handles the detection process end-to-end. It starts with raw job postings from the EMSCAD dataset (and live postings scraped from remote job sites). The information fields such as job description, company information, requirements, and benefits to capture the full idea of the posting. Further, the text is put onto RoBERTa, a transformer-based language model, to extract a 768-dimensional vector embedding from the start (CLS) token.

Because fraud job advertisements or postings are not so common (up to than 5% of the total dataset), there is a major class imbalance problem. If the model is trained on this raw distribution, it will just learn to guess "legitimate" every time to get a high accuracy score. To prevent this, the SMOTE (Synthetic Minority Over-sampling Technique) in the leftover space to generate synthetic fraud samples. Then a Multi-Layer Perceptron (MLP) neural network is trained on the balanced embeddings to classify the job listings. Finally, the SHAP values are used to break down which words in the posting contributed most to the fraud prediction, allowing to send clear, explained warnings to job seekers.

3.2 Data Collection and Preprocessing

3.2.1 Introduction to Data Collection and the Recruitment Landscape

The digitalization in the global job market has completely changed the way organizations attract talent and how job seekers look for jobs. Today, online recruitment platforms such as job boards like LinkedIn, Indeed, and Glassdoor along with dedicated job portals and remote work aggregators publish millions of jobs every day. This shift towards online recruitment has led to reduction in hiring costs and an increase in speed of hiring. But at the same time, here also, online recruitment fraud has risen due to the presence of dishonest people.

Online recruitment fraud, commonly referred to as recruitment scams, includes the publication of fake, malicious, or non-existing job advertisements. These advertisements are designed to exploit vulnerable job seekers, often individuals under financial pressure or seeking remote employment. The objectives of these fraudulent operations are diverse, encompassing:

1. Financial trap: Convincing the candidate to pay advance fees for training materials, background checks, visa processing, or dummy "work-from-home starter equipment" from a pre-approved vendor.
2. Identity Theft and Harvesting of PII: Stealing highly sensitive personal data, such as Social Security Numbers (SSNs), bank account numbers, passport details, and credit card numbers under the reason of pre-employment background checks or direct-deposit setups.
3. Money Laundering: Recruiting candidates for roles such like "financial representative" or "shipping coordinator," where they are instructed to get funds through check or wire transfer, withdraw the cash, and forward it to secondary accounts, effectively acting as money mules.
4. Phishing and Malware Distribution: Instructing candidates to download harmful application forms or software updates, putting their personal devices at risk.

Searching for these scams at large scale is highly challenging. Manual moderation of job listings is labour-intensive, slow, and impossible to scale across websites that receive tons of postings hourly. Moreover, modern scammers are highly experienced. Rather than relying on easily spottable, spam-like keyword sequences, they frequently copy-paste real job descriptions from real corporations, reference actual company officers, and match standard corporate rules.

3.2.2 The EMSCAD Dataset

The implementation is evaluated using the Employment Metadata for Spam Detection (EMSCAD) dataset, which was published by Vidros et al. in the year 2017. EMSCAD has a complete compilation of over 17,880 actual adverts for jobs. Out of these, just 866 advertisements have been nominated as fraudulent, whereas 17,014 are seen as genuine and authentic. This makes for quite an unbalanced dataset considering that around only 5% of the total dataset is fraudulent. This imbalance in the dataset resembles the real-world job environment wherein the number of scams is relatively small when compared with the number of genuine and authentic adverts. The records in the datasets contain both structured types of fields (like categorical tags and binary indicators) and free text fields.

The raw features are:

‘title’: The title of the job.

‘location’: Where the job is located.

‘department’: The division of company (e.g., Sales, Engineering).

‘salary range’: Expected salary range (mostly blank).

‘company profile’: A description of the hiring company.

‘description’: The job details and responsibilities.

‘requirements’: The skills and qualifications needed.

‘benefits’: What the company offers in return (insurance, pay, perks).

‘telecommuting’: A 0/1 flag indicating if the role is remote.

‘has company logo’: A 0/1 flag showing if the post has a logo.

‘has questions’: A 0/1 flag showing if there are screening questions.

‘employment type’: E.g., Full-time, Part-time, Contract.

‘required experience’: Entry-level, Associate, Mid-Senior, etc.

‘required education’: High school, Bachelor's, Master's, etc.

‘industry’: For example, IT, Marketing, Retail.

‘function’: For example, Engineering, Customer Service.

‘fraudulent’: 0 is the job is real, else 1.

Table 3.1: Raw features of EMSCAD

Feature Name	Data Type	Missing Values (%)	Description
Title	Text	0%	The title of specific job
Location	Text	1.93%	Country, state and city of the listing
Department	Text	64.58%	Department hosting the position
Salary range	Numeric	83.95%	Financial compensation
Company profile	Text	18.5%	Information of hosting company
Description	Text	0.01%	Information about job
Requirements	Text	15.07%	Expected skills and education
Benefits	Text	40.32%	Insurance, perks
Telecommuting	Binary	0%	Remote or on spot
Has company logo	Binary	0%	Logo presence
Has questions	Binary	0%	Screening questionnaire presence
Employment type	Text	19.41%	Full-time, par-time
Required experience	Number	39.43%	Years worked
Required education	Number	45.4%	Degree requirements
Function	Categorical	36.10%	Function category of role
Fraudulent	Binary	0.00%	Target label

3.2.3 Text Field Combination

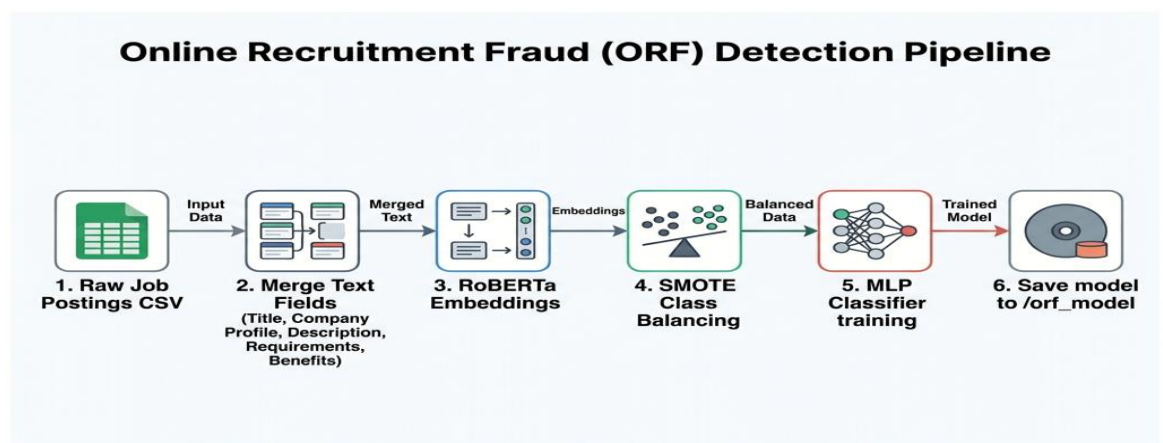


Figure 3.2 : Online Recruitment Fraud (ORF) Detection Pipeline

Majority of the text classification models only analyse the job description column. In recruitment fraud, however, looking at the description alone is not often enough. Scammers frequently copy-paste a genuine job description from a real company but put the fraudulent content in other fields. For example, the job duties may look normal, but the ‘requirements’ part might say that the candidates "must have a bank account ready to accept check deposits," or the ‘benefits’ column might mention "immediate 10,000 rupees sign in bonus paid via Telegram chat". To make sure the model looks at the entire details, we merge the important text fields into a single long string. For each job posting, we create a consolidated text sequence.

3.2.4 Preprocessing and Text Cleaning

Before providing the merged text into the language model, we clean it up:

1. Handling Missing Values: Columns containing the text data in the EMSCAD dataset often contain missing values. While merging the fields, we replace any null or missing values with an empty string (" "). This method avoids the literal words “nan” or “null” from being written into the middle of the text.
2. Whitespace Normalization: Job postings frequently contain formatting mistakes like consecutive spaces, tabs spaces, newlines, or stray HTML tags. The string is split by whitespace and join it back together with single spaces:

```
import re  
text = re.sub (r'\s+', ' ', text).strip()
```

This code written above finds group of spaces or newlines if present any and replaces them with a single space. This cleans the text, removes extra white spaces if present any, and deletes any leading or trailing spaces.

3. Filtering Empty Records: If any job advertisement becomes completely empty after the above step, that particular job advertisement is removed from the dataset so that the model does not train on any blank inputs.

3.2.5 Partitioning and Stratification

The cleaned-up dataset is then split into a ratio of 80 percent training dataset and remaining 20 percent as test dataset. Because the dataset is imbalanced highly (only around 5% of the data is fraud data), a simple random split can easily disrupt the ratio

of classes. For example, a test dataset with fraudulent postings can yield unreliable evaluation metrics. To prevent this, stratified splitting is applied. This technique makes sure that both the training and testing sets maintain the exact same ratio of legitimate to fraudulent postings as the original, full dataset. The table below details the distribution of the EMSCAD dataset. It shows how the total 17,880 samples are divided into the majority legitimate class and the minority fraudulent class across an 80-20 training and testing split.

Table 3.2 : Testing dataset and Training Dataset Split

Dataset Split	Legitimate Class	Fraudulent Class	Total Records	Fraudulent Ratio (%)
EMSCAD Dataset	17,014	866	17,880	4.84
Training Dataset (80%)	13,611	693	14,304	4.84
Testing Set (20%)	3,403	173	3,576	4.84

Table 3.3 : Top single words and consecutive words

Rank	Legitimate Postings (Single words)	Fraudulent Postings (Single words)	Legitimate (Consecutive words)	Fraudulent (Consecutive words)
1	team	money	Development team	Work home
2	experience	work	Software engineer	Data entry
3	development	earn	Project management	Make money

4	business	home	Equal opportunity	Telegram contact
5	management	telegram	Full time	Immediate start
6	support	deposit	Customer service	Quick cash
7	systems	bank	Growth opportunities	Bank account
8	software	account	Technical support	Wire transfer

3.3 Model Development : RoBERTa

3.3.1 Contextual vs. Static Word Representations

Vector space modelling in Natural Language Processing (NLP), the conversion of natural language-based text into numbered vectors so that they can be mirrored in such a way that mathematical classifiers are able to operate on them, is one of the most foundational NLP challenges. This representation was done conceptually using static vector space models. These methods include both simple frequency-based approaches and dense, low-dimensional vector representations.

Frequency-Based Static Models

Bag-of-Words(BoW): Here we prepare the document by transforming it into a high-dimensional but sparse vector where all words in the vocabulary correspond to an index and its value represents the number of times that word occurred in our document.

Term Frequency-Inverse Document Frequency: TF - IDF ranks the pure number of times a word occurs in a document with respect of its inverse across the corpus to reduce penalty on words that occur too often across all documents

Dense Static Embeddings

Many models applying the idea of word embedding and contributing to a major change appeared in the beginning of the 2010s. Popular techniques like Word2Vec with the use of two approaches like Skip-Gram or Continuous Bag-of-Words (CBOW) are significant examples alongside GloVe (Global Vectors for Word Representation) and FastText. They assign the words vector dense low-dimensional vectors (usually up to 100–300 dimensions) learned on large news collection $v_{\text{word}} = W \cdot x_{\text{one_hot}}$ is a learnt

weight matrix and one for the one hot representation of that word x_{one_hot} . Linguistic similarities are captured by these models: similar words map nearby in the vector space, and provide for mathematical analogies like $v_{king} - v_{man} + v_{woman} \approx v_{queen}$

The Polysemy Bottleneck

While static embeddings are useful, they have an important limitation which is they map each word to a single, fixed vector, means that the context is not considered while in their creation. This results in a polysemy bottleneck, where the same word has more than a single meaning depending on context, yet receives the exact same vector output representation. To say for example, consider the word "check" in two different job post scenarios:

- Scenario A (Legit): "We conduct a full background check before extending an offer."
- Scenario B (Fraud): "We'll come and send you a check in the mail to purchase supplies for your home office."

In scenario A, the word “check” is being used for the verification process. In scenario B the word refers to financial instrument through the fraud vector. If "check" in both sentence maps to the same vector, a static model such as Word2Vec leaves it up to the downstream classifier to resolve the ambiguity.

The Contextual Paradigm

Modern transformer models get around this problem by generating representations that actually depend on context, instead of using a static lookup table where a word always maps to the same vector no matter what. Here, a word's representation is computed fresh, based on the entire sentence it appears in , so the same word can end up looking completely different in vector space depending on what's around it.

Mathematically, for a word sitting at position i in a sentence of length n , its representation

h_i is a function of the whole sequence, not just itself:

$$H_i = f(t_1, t_2, \dots, t_i, \dots, t_n)$$

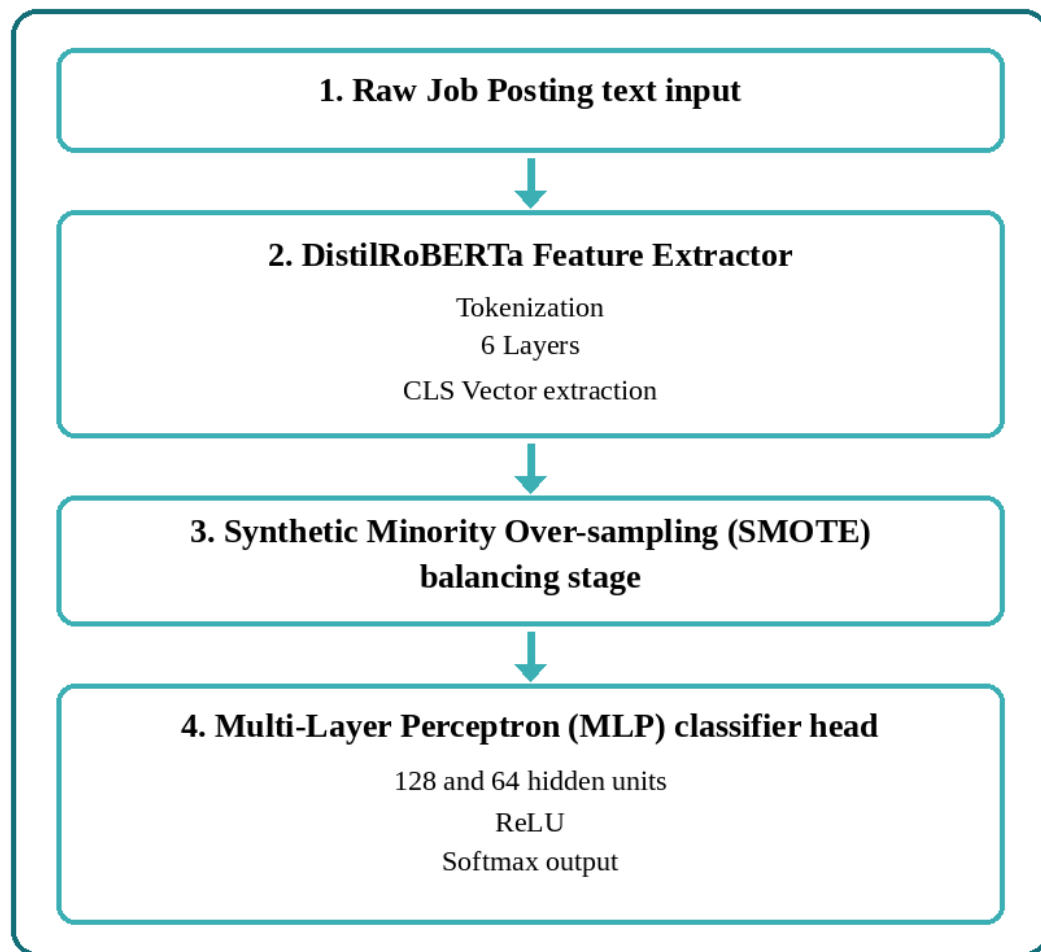


Figure 3.3: Model Development Pipeline Diagram

In other words, every other token in sentence has some way in how this one gets represented, where a word's vector never changed no matter what sentence it showed up in which is a sharp departure from older methods. This is accomplished via a special mechanism called bidirectional attention which helps the model fine-tune the representation of a certain token by allowing it to take into account its context, i.e. other tokens that come before or after it. During the process, a single token is transformed into three vectors (Query, Key, and Value) which the model relies on when determining the amount of attention of each token to other tokens in the input text.

Take "check" as the example. In the word "background check," the word "background" sits right next to it, so attention pulls "check" toward meanings tied to verification and screening, the kind of language you would expect in a legitimate hiring process. But

in "mail a check for office supplies," the neighbouring words are different totally, and instead of "check" attention pushes towards meanings tied to payments and money, exactly the kind of language that shows up in advance-fee scams. Because this recalculation happens every single time, on a sentence-by-sentence basis, the model never locks "check" into one fixed meaning. Its constantly reallocating attention based on context. Stack this across DistilRoBERTa's 6 transformer layers, and the representation gets refined again and again, earlier layers pick up basic grammar and sentence structure, while deeper layers start capturing more abstract, fraud-relevant signals. All in all, this is what allows the classifier to go beyond looking for the presence of keywords. It evolved from simply finding listings that say "check" to recognizing when "check" originates in a context synonymous with fraudulent practices, such as asking for money upfront or requesting bank account information, as opposed to showing up in commonplace hiring.

3.3.2 Contextual Feature Extraction via DistilRoBERTa

Older classification models depend on fixed numerical representations of words, like TF-IDF or Word2Vec. The problem with these approaches is that they assign a word the same vector every single time, regardless of the context it shows up in. As discussed earlier, this leads to what is called as a polysemy bottleneck, the word "check" gets represented identically whether it's part of a routine "background check" (a perfectly legitimate context) or a request to "mail a check for office supplies" (which, in this domain, often signals fraud).

This pipeline depends up on DistilRoBERTa (distilroberta-base), which is a compressed version of RoBERTa (Robustly Optimized BERT Approach) to understand what a word means in the context. DistilRoBERTa uses just 6 transformer layer instead of RoBERTa's 12 transformer layers which reduces the parameter count nearly to half, from 125 million down to 82 million. Despite this reduction in the size, it still holds onto roughly 95% of RoBERTa's language understanding ability, while running about twice as fast. That speed-accuracy trade-off is exactly why it's a good fit here, real-time web scraping and classification tasks need a model that won't slow the pipeline down.

The Tokenization Process

Before the transformer can do anything using the raw text, that text needs to be broken down into sub-word units first. We handle this process using `AutoTokenizer.from_pretrained("distilroberta-base")`. Here's what happens when the tokenizer processes a combined text sequence, t_i :

1. Byte-Pair Encoding (BPE): The tokenizer uses a byte-level BPE vocabulary of 50,265 entries which breaks the text into tokens.

It can handle typing mistakes, corporate vocabularies, and deliberately make unclear or hard to understand characters without ever falling back on an out-of-vocabulary ([UNK]) token because it works at the byte level.

2. Special Token Insertion: A sequence start token `<s>` is added at the beginning (RoBERTa's version of BERT's [CLS]), and a sequence-end token `</s>` gets added at the end (equivalent to BERT's [SEP]).

3. Padding and Truncation: Padding and truncation is done so that batches of text stay equal in size, every sequence is either padded or truncated to a fixed length of $L = 128$ tokens. Padding uses the special `<pad>` token ($ID = 1$).

4. Attention Masking: In the end the tokenizer constructs a binary attention mask by marking 1 for a real token and 0 for a padding token which is used as input to the model's self-attention layers to ensure the padding is dropped and only real information is processed.

3.3.3. The CLS Token Embedding Extraction Strategy

Once the text is tokenized, the token IDs and attention masks are then converted into input tensors and fed into `AutoModel.from_pretrained("distilroberta-base")`.

Inputs go through the 6 encoder layers of the transformer to receive outputs in the form of hidden state tensors. In the next step, we have a term `"outputs.last_hidden_state."` which refers to the embedding vector. Therefore, we can express `"outputs.last_hidden_state"` as $\mathbb{R}(B \times L \times 768)$ (1) where the properties are: (i) B = batch size (32 in this example since it fits into memory), $L = 128$ (i.e. length of sequence inputted), $d = 768$ i.e. hidden dimensions of the transformer. After that, the next line in the code

is related to obtaining first token of differences sequence which can be indexed by 0 from embeddings.

```
# Extracting the CLS embedding in train_model.py with torch.no_grad():
outputs=model(inputs)
cls_embeddings=outputs.last_hidden_state[:, 0, :].cpu().numpy()
```

Mathematically, the extraction yields a vector x_i : $x_i = h_i, 0 \in \mathbb{R}^{768}$ for a job posting i where h_i is the hidden state vector at index 0.

Because the transformer layers use the bidirectional self-attention, the hidden state of the first token takes average context from all other tokens which are in the sequence. It acts as a dense semantic summary of the entire job posting. Extracting this 768-dimensional vector allows us to compress a variable-length text document into a fixed-size vector that captures the document's meaning.

3.3.4. Classification Head: The Multi-Layer Perceptron (MLP)

The 768-dimensional embedding vectors which is extracted x_i are used as input features for the classification head. It is implemented by means of the MLPClassifier class from the scikit-learn library. While linear models like Logistic regression or SVM models assume linear connections and models such as the Random Forest make use of axis-aligned splitting in the space, MLP works with hidden layers and activation functions which are not linear.

Network Architecture:

The structure of the MLP model defined in the training script (train_model.py) is:

- The Input layer: 768 nodes (the CLS vector).
- The 1st hidden layer: 128 neurons.
- The 2nd hidden layer: 64 neurons.
- The output layer: 2 nodes (the classes are Legitimate and Fraudulent).

The equations showing the forward propagation of the inputs through the MLP are as follows:

1. First Hidden Layer: $z^{(1)} = x W^{(1)} + b^{(1)}$

$$a^{(1)} = \text{ReLU}(z^{(1)}) = \max(0, z^{(1)})$$

2. Second Hidden Layer: $z^{(2)}=a^{(1)}W^{(2)}+b^{(2)}$

$$a^{(2)} = \text{ReLU}(z^{(2)})=\max(0, z^{(2)})$$

3. Output Layer: $z^{(3)}=a^{(2)}W^{(3)}+b^{(3)}$

$$\hat{y}=\text{Softmax}(z^{(3)})=[\hat{y}_0 \ \hat{y}_1],$$

where:

- $W^{(1)} \in \mathbb{R}^{768 \times 128}$ and $b^{(1)} \in \mathbb{R}^{128}$: represents the weights and biases of the first layer.
- $W^{(2)} \in \mathbb{R}^{128 \times 64}$ and $b^{(2)} \in \mathbb{R}^{64}$: represents the weights and biases of the second layer.
- $W^{(3)} \in \mathbb{R}^{64 \times 2}$ and $b^{(3)} \in \mathbb{R}^2$: represents the weights and biases of the final classification layer.
- $\hat{y}_0$ and $\hat{y}_1$: respectively represents the predicted probabilities of the listing being legitimate and fraudulent.

The Rectified Linear Unit (ReLU)

One of the activation functions employed in the hidden layers is the Rectified Linear Unit (ReLU) which is defined as $f(z) = \max(0, z)$ its computational simplicity contributes to eliminating the gradient saturation problem. For any positive input, its gradient is 1, which allows error gradients to propagate back through the network layers during training without vanishing.

3.3.5. Training, Optimization, and Early Stopping

Loss Function and Optimizer

The Cross-Entropy loss function is used to optimize the network, which measures the difference between the predicted probabilities $\hat{y}_i$ and the ground-truth binary labels $Y_i \in \{0, 1\}$: $L=-(1/M) \sum_{i=1}^M [y_i \log(\hat{y}_i,1)+(1 - y_i) \log(\hat{y}_i,0)]$ where M is the size of the training batch.

To reduce the amount of this loss, the classifier uses the ADAM (Adaptive Moment Estimation) optimizer. ADAM updates the weights by computing adaptive learning rates for each parameter, tracking both the gradient's mean (first moment) and variance (second moment) over training iterations.

Overfitting Control using Validation-Based Early Stopping

Because the MLP is trained on embeddings balanced via SMOTE (which generates synthetic data by interpolating between existing minority samples), there is a risk that the model will overfit to these synthetic patterns and generalize poorly to real-world data.

To prevent this from happening, our training script: configures `early_stopping=True` and `validation_fraction=0.1`:

```
# MLP configuration in train_model.py
clf=MLPClassifier
(
    hidden_layer_sizes=(128, 64),
    max_iter=200,
    random_state=42,
    early_stopping=True,
    validation_fraction=0.1
)
```

The early stopping mechanism works as described:

1. Validation Split: MLP class saves the 10% of training embeddings before the training starts.
2. Monitoring: The loss is calculated on this validation set at the end of each training.
3. Termination: The training stops, if there is no improvement in validation loss (at least 10- improvement as defined by the parameter) during 10 iterations.
4. Weight Restoration: The optimizer stops and all the weight matrices that produced the minimum validation loss are restored ultimately.

3.3.6. Inference and Serialization Workflow

Once training process is complete, the models and their configurations are saved to a directory (`./orf_model`) for deployment.

Serialization (Saving the Pipeline)

To ensure the model can be loaded for conclusion, the training script saves:

1. The Tokenizer: Saved using `tokenizer.save_pretrained(save_dir)`. This preserves the byte-level BPE vocabulary and config files.

2. The configuration of the Base Model: Saved using `roberta_model.config.save_pretrained(save_dir)`. This ensures that the conclusion made on based facts and clues engine knows the structure of the base model.
3. The Classifier Head: The trained `MLPClassifier` contains the learned weight matrices ($W^{(1)}$, $W^{(2)}$, $W^{(3)}$) and bias vectors. We serialize the classifier to a single file using `joblib.dump(clf, clf_path)`.

Deserialization (Inference Pipeline)

Inference server (app. A model is loaded into memory py or `predict_and_alert.py`) as follows:

```
tokenizer=AutoTokenizer.from_pretrained(save_dir)
roberta_model=AutoModel.from_pretrained(save_dir)
mlp_classifier=joblib.load(os.path.join(save_dir, "classifier.joblib"))
```

For example, when a user creates a new job posting or a scraper online pulls an active ad from the internet, this server retrieves relevant text and processes it to obtain the CLS embedding in 768-dimension which will be sent to an MLP classifier in order for predictions of class probabilities (normal/fraud) to be made.

3.4 SMOTE (Synthetic Minority Oversampling Technique) Framework

3.4.1 The challenge of Class Imbalance

Most large and supervised learning algorithms are built on an assumption that is not actually true in the real world, that the training data comes from a roughly balanced set of data classes. In real world classification tasks such as credit card fraud detection, medical anomaly detection, or ORF, the target minority class is significantly less than majority class.

1. The Accuracy Paradox

If 95.16% of the dataset has legitimate listings, a machine learning model trained only on raw accuracy can achieve a 95.16% simply by saying legitimate for every input. The model gets a high accuracy without actually learning any separate patterns

of features belonging to fraud class. Looks impressive on paper, but its recall on the fraud class is literally 0%, meaning it catches zero fraudulent postings.

2. Loss and Gradient Bias

Standard loss functions, such as Binary Cross-Entropy calculate the average error across every training examples. When the majority class outnumbers minority class, the model calculates its gradient mostly based on the majority examples during backpropagation. So, the model's weights keep adjusting to fit legitimate postings, while the patterns that actually define fraud barely register.

3. Decision Boundary Skew

This imbalance also shifts the model's decision boundary very close to the fraud class. Because there are less fraud examples available, the model cannot study all the scam variations. This causes the system to miss new types of fraud, leading in high false negative rates.

To fix this, researchers balance the dataset before training. Two common ways are:

- **Random Under-Sampling (RUS):** This method drops majority class examples till the classes become equal. It is easy to run, but it throws away real data. Losing real postings means the model learns a smaller vocabulary, making it tough to spot phishing attempts.
- **Random Over-Sampling (ROS):** This method copies the existing fraud examples instead of deleting files. No data is lost, but no new patterns are added either. The model simply memorizes the same fraud examples, which cause poor performance on new scams.

3.4.2 Theoretical foundations of SMOTE Applications

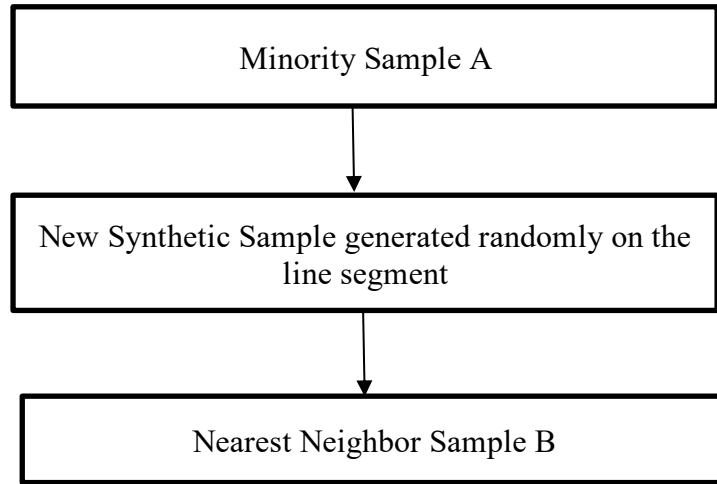


Figure 3.4: SMOTE Interpolation

Here is step by step how the algorithm works:

Step 1: Distance Calculation

For each and every minority-class sample x_i the algorithm measures its distance to all the other minority-class samples x_j , using standard Euclidean distance:

$$d(x_i, x_j) = \sqrt{\sum_{d=1}^D (x_i[d] - x_j[d])^2}$$

here D indicates the quantum of dimensions in the feature space which means the 768-dimensional contextualized embeddings extracted from the transformer model. Hence the value of D is equal to 768.

Step 2: Nearest-Neighbor Selection

After the distances have been calculated, the algorithm proceeds to calculate the k -nearest neighbors for each instance x_i . This pipeline uses the value $k = 5$ since this is the most widely used default value. Keeping k at this size matters, it keeps the interpolation confined to tight local clusters within the minority class, rather than accidentally reaching into regions of the feature space that are dominated by legitimate (majority-class) postings.

Step 3: Interpolation and Generation

Now one of the k neighbors, denote it x_{nn} , is selected at random and a new synthetic example x_{new} will be created at some point in the line connecting it to x_i :

$$X_{new} = x_i + \lambda (x_{nn} - x_i)$$

Where λ is defined as a random number generated according to the uniform distribution:

$$\lambda \sim U(0, 1)$$

which is what determines exactly where along that line segment the new synthetic point ends up landing.

This looping process continues to generate synthetic data points. It repeats until the minority class matches the size of majority class. This setup stops the model from favouring real postings during training.

3.4.3 SMOTE in Continuous Vector Space v/s Discrete Vector Space

Oversampling can be applied either to the raw text or to its vector representations. However, using SMOTE on raw text fails due to the discrete natural language.

1. Discrete Token Limits

Text contains of different tokens from a vocabulary dictionary, with no numerical values between them. Consequently, it is mathematically impossible to calculate an average point between two words. For example, trying to find a midpoint between “Telegram” and “WhatsApp” does not produce a valid word.

2. Broken Grammar

The usual ways people try to increase text, changing same meaning words, deleting random words, back-translation, tend to produce sentences that don't quite make grammatical meaning anymore. That's a problem for transformer models specifically, since they rely on natural sentence structure to compute self-attention properly. If these kind of broken sentences are fed, the performance of the model suffers.

3. Meaning Can Flip Entirely

Changing the sentence structures may also lead to significant alterations in meaning as it can be risky in the area of fraud detection where even a slight difference between the phrases "verified check" and "fake check" means that the message became fraudulent instead of being legal, adding misrepresented noise to the training data. Thus, it is preferable to avoid using raw text in the pipeline. This gives us a few real advantages:

- Interpolation actually means something here. In embedding space, position corresponds to meaning, vectors that are close together represent similar concepts. So, when SMOTE interpolates between two fraud vectors, the synthetic vector it creates genuinely represents a blend of both, say, mixing traits of an upfront-payment scam with traits of a fake remote-data-entry job.
- Grammar stays intact. Since oversampling happens after the text has already passed through the transformer, none of the synthetic samples touch the actual sentence structure. The transformer only ever sees real, unmodified sentences, the synthetic vectors exist purely downstream, in number form.
- It's computationally cheap. Generating synthetic vectors is just basic linear algebra, no need to reprocess text or run the transformer again during oversampling, which keeps the whole pipeline fast.

3.4.4 Balancing Configuration and Class Distance Analysis

The entire EMSCAD dataset is divided into two parts, one is the training dataset which contains 80% of the data, rest 20% is put into the testing dataset. The SMOTE algorithm is only applied on the training set to make sure that the class imbalance does not bias the model evaluation.

Table 3.4 : Class Distribution of Training Sets and Testing Sets

Dataset Split	Legitimate Class (y=0)	Fraud Class (y=1)	Total Records	Fraudulent ratio (%)
Original Training Set	13,611	693	14,304	4.84
Balanced Training Set	13,611	13,611	27,222	50.00
Testing Set	3,403	173	3,576	4.84

The Importance of Testing Set Integrity

One rule that has to be followed strictly: SMOTE never gets applied to the testing set. The reasoning here is straightforward, the testing set needs to mirror what's actually happening in the real world, where fraudulent job postings are genuinely rare compared to legitimate ones. On the other hand, if we also applied balancing to the testing set then the values we obtain for such metrics as precision, recall, etc. would also be inflated artificially, thus giving us a misleading impression about the performance of the model once it's actually launched. By intentionally leaving the testing sample imbalanced, we can be sure that the findings of validation reproduce the real performance of the model in the production environment and not the best-case scenario that doesn't exist in practice.

3.4.5 Theoretical Impact of SMOTE on Classifier Decision Boundaries

Applying SMOTE to balance the training embeddings doesn't just change the number of samples in each class, it actually reshapes how the classifier differentiates between "legitimate" and "fraudulent." Here's what's happening under the hood:

1. The Minority Class Gets More Room to Breathe

In an imbalanced dataset, fraud examples are less, scattered thinly across the feature space, with huge empty gaps between them. A classifier trained on data like this has almost nothing to learn from in those gaps. It simply hasn't encountered enough instances of fraud to be aware of what exactly counts as "fraud." SMOTE addresses that by filling in the gaps with synthetic points and extending the boundaries of the

fraud class for the model. Instead of isolated dots representing fraud cases, the model gets the complete representation of the fraud domain in terms of features.

2. The Decision Boundary Shifts to Where It Should Be

Without balancing the dataset, the large volume of legitimate listings drowns out the loss signal coming from fraud examples, the model's optimization is dominated by whatever's most common. Once SMOTE balances the class counts, this imbalance in the loss function disappears. The result is a decision boundary that shifts away from where it would otherwise get pushed, right up against the sparse fraud cluster, and moves toward a fairer midpoint between the two classes. Practically speaking, this means fraud listings that would have previously slipped through and gotten classified as "legitimate" now fall on the correct side of the boundary.

3. A Side Benefit: Better Generalization

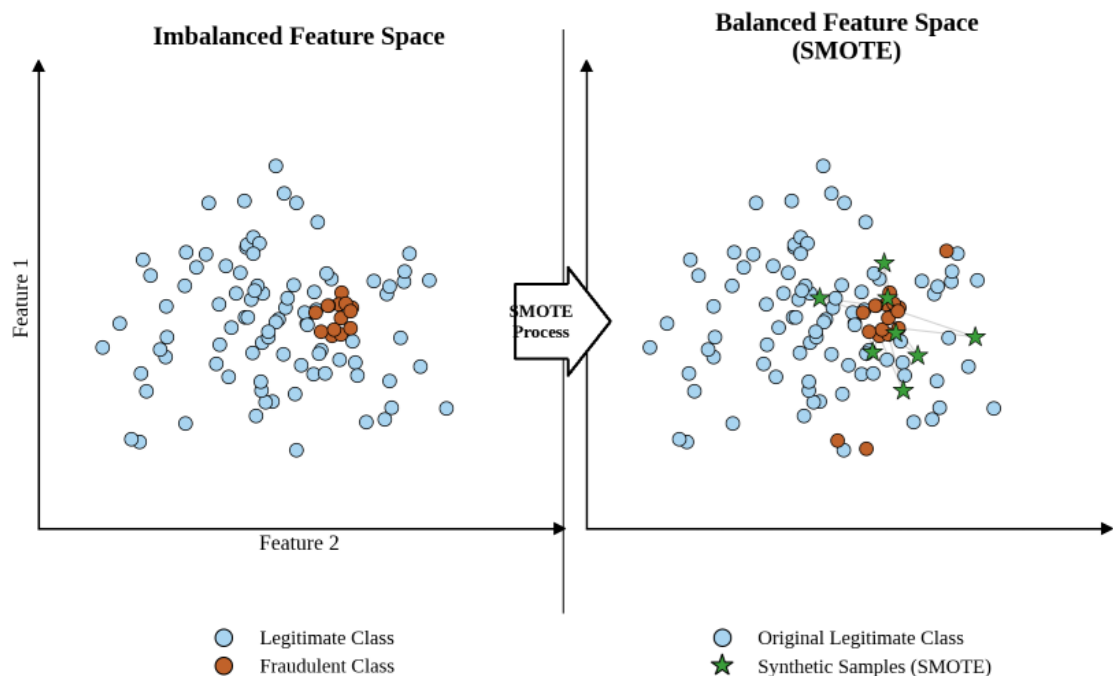


Figure 3.5: SMOTE Vector Space Balancing Diagram

Because SMOTE doesn't just duplicate existing fraud samples but generates new, slightly different synthetic ones through interpolation, it introduces a bit of natural

variation into the fraud class. This turns out to work almost like a built-in regulariser, instead of the classifier memorizing exact training examples (which would make it great at spotting fraud it's already seen but useless against anything new), it's nudged toward learning the broader pattern of what fraud tends to look like. That's exactly what you want for a real-world detection system, since new job scams will rarely look identical to the ones already in the training data, the model needs to generalize, not memorize.

3.5 Experimental Evaluation Techniques

3.5.1 Rationale for Evaluation in Imbalanced Class Specifier

Once a model is built, the real question is whether it is doing what it is actually intended to do, and for a problem like this, that is trickier to answer than it sounds. Most evaluation habits assume the two classes you're predicting are roughly balanced. That assumption breaks down badly for something like online recruitment fraud detection, where fraudulent postings make up just 4.84% of the dataset.

Here's why that matters: a model that never bothers to flag anything as fraudulent, that just labels every single job posting "legitimate", would still be right 95.16% of the time. On paper, that looks like a strong result. In practice, it's useless. It wouldn't catch a single fraud data, which means its recall on the thing we actually care about is zero. Accuracy, in other words, can lie to you when the classes are this skewed.

Therefore, rather than relying on accuracy, the assessment here revolves around the use of the confusion matrix which analyzes predictions into four classification options on the basis of model predictions vs. actual results.

1. True Positives (TP): the fraudulent listing identified successfully by the model.. This is the win condition: the system flags it, and a security alert goes out before a candidate gets hurt.
2. True Negatives (TN): a legitimate listing correctly recognized as legitimate. Nothing exciting happens here, which is exactly the point, the posting goes live and recruiters and candidates go about their business undisturbed.
3. False Positives (FP): a legitimate listing that gets wrongly flagged as fraud. This one's costly in a quieter way: too many of these and analysts start tuning out the alerts ("alert fatigue"), and real recruiters end up fighting the system just to post a genuine job.

4. False Negatives (FN): a fraudulent listing that slips through as "legitimate." This is the one that should worry you most. It's the failure mode where the scam actually reaches people, opening the door to financial fraud, identity theft, or money laundering.

Framing the evaluation around these four outcomes gives a much more honest read on the pipeline, not just how often it's right overall, but how it fails, and how badly each kind of failure would hurt if this were running for real.

3.5.2 Core Performance Metrics: Precision Recall, and F1-score

Precision:

Precision indicates the accuracy of prediction! What proportion of all the content and spam, was actually fraudulent in the content flagged?

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

High precision means when you, the model, raises a flag, you'd usually believe it, it isn't crying wolf. That stops real jobs from being blocked or delayed simply because the model got a little jittery.

Recall:

Recall is about completeness. Then, of all the real fraud that exists, how much did the model discover?

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

With high recall, the system doesn't leave much on the table. This is what really matters for candidate safety, a false positive that the model throws up isn't just a bad point of data, this is someone who could actually lose money or have their ID stolen.

Why you can't have maximum both?

Precision and recall tend to be at odds with one another, and the key thing that controls that battle is the decision threshold you set for flagging something as fraud.

Lower the threshold and the model becomes more aggressive, it flags more listings. This raises recall since more actual scams are caught along the way, but it also brings in false positives, so precision goes down.

Increase the threshold, and your model becomes more conservative, only makes a prediction where it is relatively confident. Since the flags are cleaner, precision

improves but some of the sneakier, more subtle scams start to get waved through so recall suffers.

Abstractly speaking, neither setting is "correct". Ultimately, it boils down to the false positive tolerance you can accept, and in a fraud-detection setting, this is rarely an easy question to consider.

F1-Score:

For a single score that mixes both numbers for how well the model is balancing this trade off we can use the F1-Score, the harmonic mean of precision and recall:

$$\text{F1-Score} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

Note that the harmonic mean is being used here, not an average, a plain average can mislead you. If we had a model that had 100% precision but 0% recall (where the model never falsely accuses anyone, but also never once catches a scam), it would still be summarised as averaging out to something like a respectable-looking 50%. This is driven home by the harmonic mean; it won't? So what? At best, the skinny model drops right down to 0%, and there's much less humbug in a measure that can produce such an 'honourable' - if that's even the word for it - result from such dishonour. And that's what F1 is all about: a singular number that prevents a model from hiding behind good performance on just one side of the trade-off.

3.5.3 Threshold Independent Evaluation: PR-AUC v/s ROC-AUC

Usually, Precision, Recall, and F1 are obtained at just one threshold (usually set to 0.5). However, this is not enough to know how the model performs. One must assess model performance at each possible threshold, which is the purpose of AUC.

ROC-AUC

The ROC curve shows relationship between Recall or True Positive Rate and False Positive Rate for all possible threshold values. The False Positive Rate is defined as below:

$$\text{FPR} = \text{FP} / (\text{FP} + \text{TN})$$

Therefore, the area under the curve is in the range from 0.50 which means tossing a coin to perfect separation at 1.00. This common metric fails when applied to heavily imbalanced data. Because the legitimate class contains 17,018 samples, the number of

True negatives is high. This large base number hides the impact of False Positives in the calculation, keeping the False Positives Rate artificially low. As a result, the ROC-AUC score can look excellent even if the model performs poorly and generates a high number of false alarms.

PR-AUC

Consequently, the PR-AUC serves as a better alternative. The PR curve plots Precision against Recall across every threshold, and the area under it gives us:

$$\text{PR-AUC} = \text{Average Precision across all Recall levels}$$

The key difference is that PR-AUC never looks at True Negatives at all, it's entirely focused on how the model handles the minority class, which is the fraud cases we actually care about catching.

3.5.4 Interpretability and Explainability: SHAP Values

Deep neural networks, including the combined DistilRoBERTa and MLP architecture, work as black box models. In recruitment security, system operators must identify the specific reasons behind a fraud prediction. This study uses SHAP (Shapley Additive exPlanations), a framework based on cooperative game theory.

The Game Theory Metaphor:

In cooperative game theory, the Shapley value is a method for distributing the total payout of a cooperative game fairly among its players based on their individual contributions.

In our pipeline:

1. The Game: The prediction process where the model evaluates a job posting.
2. The Players: The individual words (tokens) in the job posting text.
3. The Payout: The difference between the model's predicted probability of fraud and the average baseline probability of the dataset.
4. The Contribution: The Shapley value calculated for each word, indicating its impact on the prediction.

Mathematical Concept:

The Shapley value (ϕ_i) for a specific feature (word) i is calculated by measuring its marginal contribution when added to various subsets of other features:

$$\phi_i = \sum \left[\frac{|S|! \cdot (N - |S| - 1)!}{N!} \cdot [v(S \cup \{i\}) - v(S)] \right]$$

where:

- N represents the set of all input features (all words in the job listing).
- S is a subset of features excluding word i .
- $|S|$ is the number of features in subset S .
- $v(S)$ is the model's output probability when only the features in subset S are active.
- $v(S \cup \{i\}) - v(S)$ represents the marginal change in predicted probability when word i is added to the subset.
- $(|S|! * (N - |S| - 1)! / N!)$ is a scaling factor that weights the marginal contribution by the number of possible permutations of the subset.

By calculating this value for every word in a job posting, the system determines which terms contributed to the fraud classification (positive SHAP values) and which terms supported a legitimate classification (negative SHAP values).

3.5.5 Text Partition Explainer for Scalable Word Attributions

Instead of evaluating combinations of individual words, the explainer calculates Shapley values across the partitions of this hierarchical tree. If a large block of text (like a standard company description) has little impact on the prediction, the explainer evaluates it as a single unit, bypassing the need to compute contributions for each word within that block. This optimization reduces the computational complexity from exponential time to polynomial time, allowing the system to generate word attributions in real time.

The Masking Mechanism

Because the underlying model (DistilRoBERTa + MLP) expects a fixed-size vector input, the explainer cannot simply delete words to measure their absence. Instead, it perturbs the input by masking words.

During evaluation, the explainer replaces target words with a neutral masking token (specifically the tokenizer's padding token, `<pad>`). It then queries the prediction pipeline to observe how the absence of those words affects the predicted fraud probability. These changes in probability are aggregated to compute the final word attribution scores.

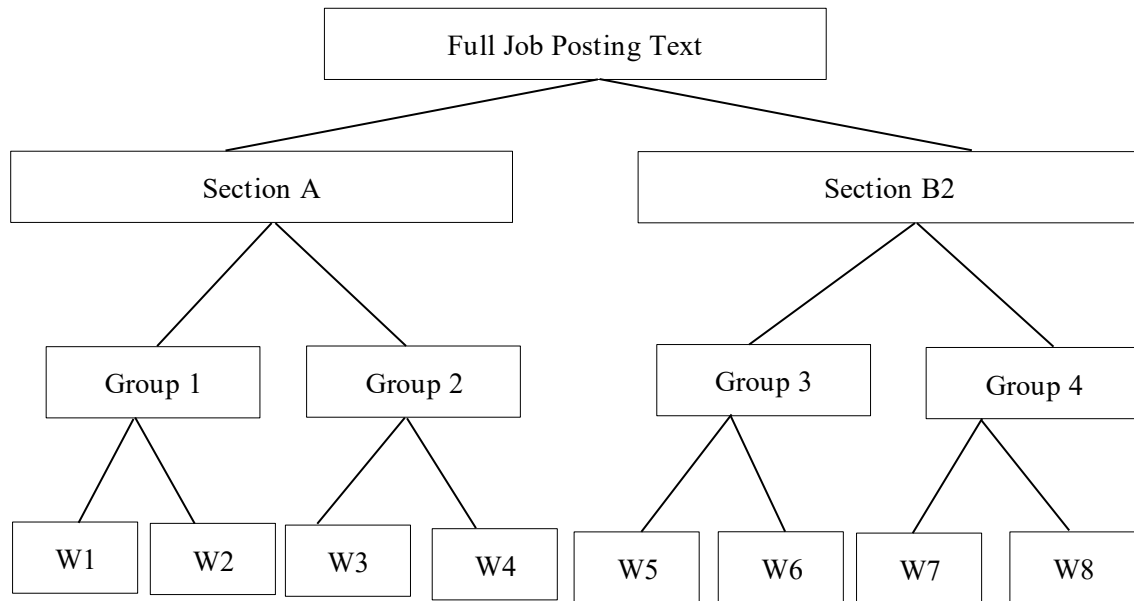


Figure 3.6: Hierarchical partitioning

3.5.6 Alert Generation and Warning Synthesis

The final component of our evaluation and deployment framework is the alert generation module. This module translates the mathematical output of the classification head and the interpretability framework into actionable security warnings for candidates.

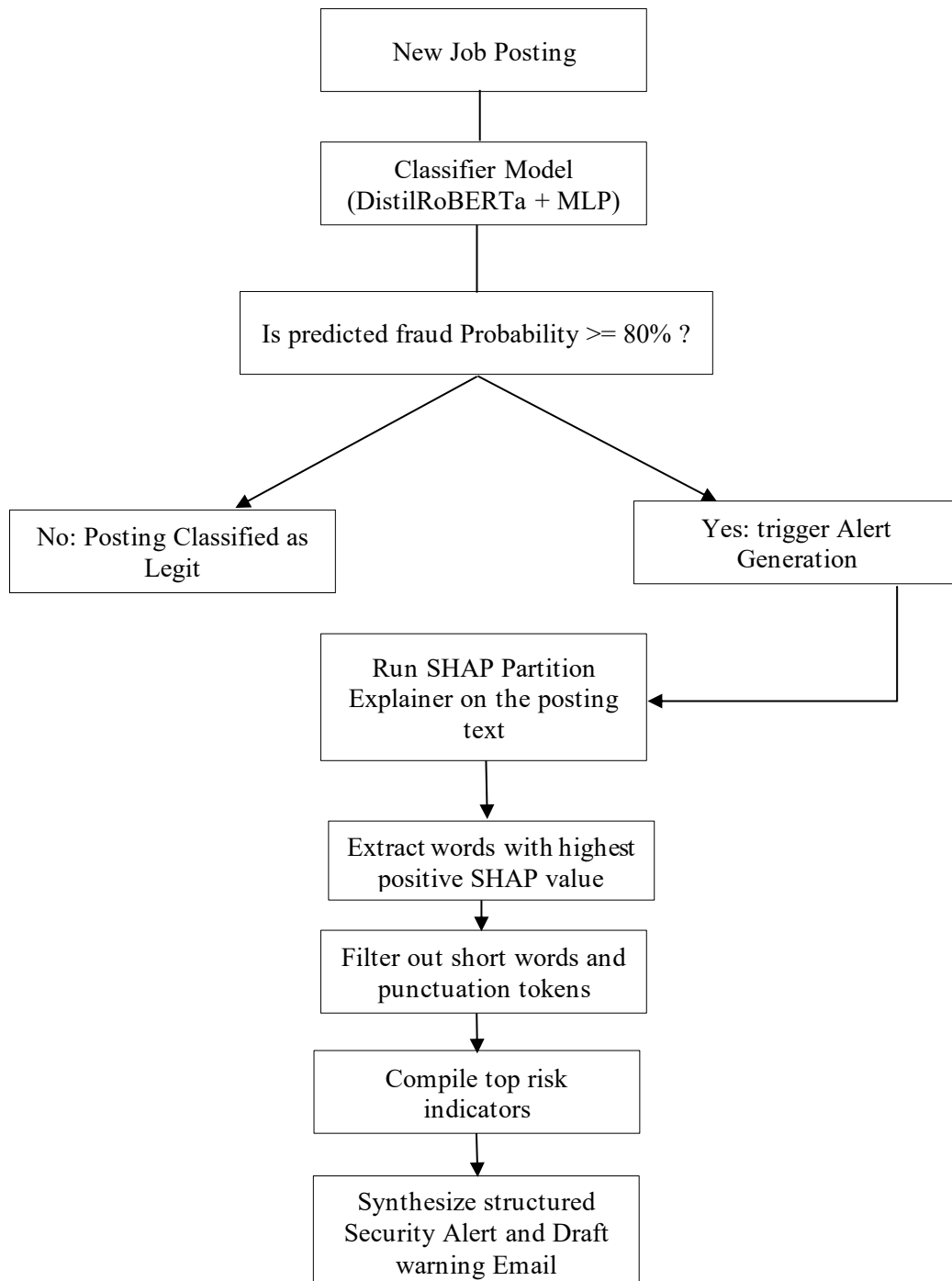


Figure 3.7: Triggering Alert Generation

The final component of our evaluation and deployment framework is the alert generation module. This module translates the mathematical output of the

classification head and the interpretability framework into actionable security warnings for candidates.

When a job posting is processed by the pipeline, the MLP classification head outputs a predicted fraud probability score. If this score exceeds a defined threshold (configured as 80% in our system), the alert generator is triggered:

1. SHAP Attribution Run: The system runs the SHAP Partition Explainer on the posting text to compute word-level attribution scores.
2. Linguistic Filtering: The raw tokens are filtered to remove punctuation, numbers, and short words (fewer than 3 characters) to focus on meaningful terms.
3. Risk Word Extraction: The system extracts the top words with the highest positive Shapley values representing the primary indicators of fraud (e.g., "Telegram", "WhatsApp", "wire transfer", "deposit").
4. Warning Synthesis: The extracted keywords are compiled into a structured security warning message. This warning is embedded within a draft notification email sent to the applicant, detailing the flagged risk indicators and providing safety recommendations to help them avoid the scam.

3.6 Summary of Methodology

1. Introduction and Background

Hiring has moved online, and it has moved fast. Between LinkedIn, Glassdoor, Indeed, Remote.co and dozens of other platforms, millions of job listings are live at any given time. This shift has been genuinely good for the labour market, cheaper to post, faster to fill roles, easier for people anywhere to find work. But it has also opened a door that scammers have been walking through with increasing confidence, and that's what this thesis is really about: Online Recruitment Fraud, or ORF.

At its core, ORF is just a fake or malicious job listing made up to look real. The people behind it aren't confused about who they're targeting, they go after job seekers who are eager, remote-work hopefuls, or people who are financially stretched and more likely to overlook red flags because they need this to work out. And what they're after usually falls into one of three buckets:

- Getting money directly: Convincing someone to pay for a background check, a visa, "training materials," or home-office equipment from some vendor that conveniently only the scammer can point you to.
- Stealing identities: Asking for Social Security numbers, bank details, passport scans, credit history, all framed as routine "pre-employment screening."
- Laundering money through people: Hiring someone as a "financial representative" or "shipping coordinator" and having them receive funds, pull out the cash, and wire it elsewhere. The person doing this often has no idea they've just become a money mule.

The obvious fix, have someone review every listing. Job boards get tens of thousands of postings an hour, and no moderation team on earth keeps up with that manually. Making it

worse, today's scammers aren't sloppy. They lift real job descriptions from real companies, drop in real executive names, and use the same corporate language everyone else uses. A keyword filter searching for the words "urgent" or "no experience required" won't help with this thing. The goal of the new system is to create a device that can identify fraud and give reasons on that.

2. Problem Statement and Core Objectives

Two problems make this harder than a typical classification task. First, the imbalance problem. Scam listings are rare. In the dataset used here, they're only 4.84% of everything. That sounds like a small technical detail, but it quietly wrecks naive approaches to the problem. A model that just labels everything as legitimate, no thinking required, would be right 95.16% of the time. This data looks impressive, but it is useless because it is not able to show any fraud case. If one wants an efficient system that does something valuable, he has to make sure it sees enough cases of both types to understand the differences between them, and does not use statistics to pick the "safest" option of response.

Second, the black-box problem. The kind of models that are good at this, deep learning, transformer-based, are also the kind that don't explain themselves. And "trust me" isn't good enough in a security context. If a job seeker or an analyst sees a listing flagged as fraud with zero explanation, they've got no way to check whether

that's right, and no reason to believe it. That kind of silence breeds either blind trust or complete dismissal, neither of which is what you want from a security tool.

So the work here is organized around three things:

1. Build a feature extractor that actually reads the listing the way a suspicious human would, picking up on subtle patterns across different fields, not just scanning the description in isolation.
2. Fix the imbalance properly, by generating synthetic examples in the model's embedding space rather than just duplicating existing ones or hoping the model figures it out.
3. Make the whole thing explainable, using ideas borrowed from game theory to figure out which specific words pushed the model toward "fraud", and turn that into a warning a person can actually read and understand.

3. Research Methodology and Data Preparation

The training and testing here relies on EMSCAD (Employment Metadata for Spam Detection) which contains 17,880 real job postings, 17,014 legitimate and 866 confirmed scams. Each job listing contains structured fields such as location, requirements, and binary flags alongside unstructured fields, including the company profile, description, and benefits.

Analysing the dataset shows that missing information provides a strong signal for fraud detection. Approximately 85% of fraud listings omit the company profile, compared to only 15% of legitimate posts, as bad actors avoid traceability. Salary fields also show different patterns between the two classes. During preprocessing, empty fields are converted to empty strings rather than deleted. This step ensures that model can process the absence of data as a separate feature.

Pulling in live listings. To see how this holds up against real, current job postings rather than just the historical dataset, a scraper was built to crawl Remote.co rotating user agents, spacing requests out by 1–2 seconds so it isn't hammering the site, then parsing the scraped HTML into the same schema the model was trained on and running it through the classifier.

Reading the whole listing at once. A lot of models just look at the job description and call it a day. But scammers are sneaky about where they hide the actual ask, the description might read completely normal, and then the "requirements" section

casually asks for a bank account number. So instead of treating each field separately, everything gets glued into one sequence:

$T = \text{title} + \text{company_profile} + \text{description} + \text{requirements} + \text{benefits}$

That way the model isn't just reading the description, it's reading the whole listing as one connected story, which is exactly where the inconsistencies tend to show up.

4. Model Architecture and Training Process

The system works in two stages: first understand the text, then decide.

Stage 1: Understanding the text (DistilRoBERTa). The combined text is processed through Byte-Pair Encoding and uses a vocabulary consisting of 50,256 tokens, enabling it to deal with typos and the little character tricks that scammers use to deceive filters without giving up because of issues with unfamiliar words. After that, each sequence is either padded or trimmed to 128 tokens and put through DistilRoBERTa, a 6-layer transformer.

What we actually pull out is the embedding for the very first token — the CLS token:

$$x = \text{Encoder}(T)[0, :]$$

Owing to the functionality of self-attention, a single token has the potential to absorb context from all other tokens present in the input sequence. Therefore, one 768-dimensional vector does not represent merely "the first word" but is in fact a compressed summary of the entire series of tokens in the input. Fixing the imbalance (SMOTE). Rather than just copying fraud examples to pad out the minority class, the training embeddings get balanced using SMOTE. For each fraudulent embedding, it finds its 5 nearest neighbours (by Euclidean distance) among other fraud embeddings, and invents new synthetic points along the line between them:

$$x_{\text{new}} = x_i + \lambda \times (x_m - x_i)$$

where λ is just a random number between 0 and 1. This takes the training set from a lopsided 17,880 up to a balanced 27,222 — 13,611 real legitimate embeddings and 13,611 synthetic fraud embeddings. Importantly, this only touches the training data. The test set is left exactly as unbalanced as reality is, so the evaluation isn't lying to us about how well this would actually work.

Stage 2: Making the call (MLP). The balanced embeddings get passed into a small feedforward network: 768 inputs $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 2 outputs (Softmax). The hidden layers use ReLU:

$$\text{ReLU}(z) = \max(0, z)$$

Training uses Adam and cross-entropy loss. And because SMOTE-generated data can lead a model to over-learn patterns that don't really exist, early stopping is used as a guardrail: 10% of the pre-SMOTE training data is held back purely for validation, and if validation loss doesn't improve for 10 straight epochs, training stops and the best weights get restored.

5. Explainability Framework and Warning Synthesis

Flagging something as fraud isn't the end of the job. The system also needs to say why. That's where SHAP comes in.

The core idea. SHAP treats each word in the listing like a player in a cooperative game, where the "payout" is the model's predicted fraud probability. The Shapley value of a word determines the actual amount by which it will change the respective probabilities, averaged over all possible combinations of words.

$$\phi_i = \sum [(|S|! \times (N - |S| - 1)! / N!) \times [v(S \cup \{i\}) - v(S)]]$$

In this case, N is the set of all words, S is a subset that does not contain word i and $v(S)$ is the result from the model produced using the subset only.

Making it actually computable. Calculating that exactly would mean checking every possible subset of words, which explodes computationally almost immediately. So instead, the pipeline uses SHAP's Partition Explainer, which clusters words into a hierarchy based on how closely related they are, then tests importance by masking words out (swapping them for a neutral padding token) and watching how much the prediction shifts without them. It aggregates all of that into a per-word attribution score, fast enough to run in real time.

Turning it into a warning. If a listing crosses an 80% fraud probability, the system runs the Partition Explainer, strips out punctuation and filler words, and pulls out the words that pushed hardest toward "fraud" things like "Telegram," "deposit," "wire transfer," "WhatsApp." Those get compiled into a warning message and dropped into a draft email that goes back to the applicant, so they're not just told "this is a scam", they're shown exactly what tipped it off.

6. Results and Strategic Implications

How we judge it. The main metric here is PR-AUC, not the more familiar ROC-AUC. The reason comes back to the imbalance problem from Section 2: ROC-AUC

can look great purely because there are so many true negatives sitting in the denominator, even if the model is throwing out plenty of false alarms. PR-AUC doesn't get fooled that way. It's entirely focused on the minority class, so the moment the model starts crying wolf, precision drops and PR-AUC drops with it.

What actually happened. Combining DistilRoBERTa, SMOTE, and MLP with early stopping results in a significant PR-AUC score on genuinely imbalanced test data, indicating that the pipeline is able to capture real scams while avoiding the interruption of legitimate listings due to false alarms.

Why this matters beyond the numbers. A few concrete payoffs fall out of this:

- Job seekers get an actual reason, not just a red flag, seeing "this was flagged because of 'Telegram contact' and 'wire transfer'" is something a person can act on.
- Platforms get relief. Job boards can plug this in and cut down the manual moderation workload rather than relying purely on human reviewers to keep up with an impossible volume.
- Security teams get something they can check. Because every decision comes with the words that drove it, analysts can actually audit the system instead of just trusting a black box, which is, ultimately, the whole point of building it this way.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Introduction

By this point, the pipeline has been designed and built, now comes the part where it actually has to prove itself. Recruitment scams today don't look like scams. They're deliberately built to pass as ordinary job postings, and by the time a victim realizes what's happening, they've already handed over a bank detail, paid a "processing fee," or unknowingly moved money for someone else. In the past, keyword-based filters served as a temporary solution to address such issues, but they are no longer effective since scammers now make use of legitimate business text, which fools any searches for odd phrasing.

This is exactly the issue that has led to the creation of the pipeline: DistilRoBERTa processes every ad and converts it into a context embedding, SMOTE balances the training data in this embedding space and MLP makes a fraud/no fraud decision while SHAP identifies the reasons for this decision being made. This chapter is where all four of those pieces get checked, individually and together.

Instead of settling for a single headline accuracy figure, the aim here is to poke at the system from several directions, does it classify well, does SMOTE actually help or just look like it does, does performance hold up once you stop picking one arbitrary threshold, do the SHAP explanations make sense to an actual person reading them, and probably the most honest test of all, does any of this survive contact with listings scraped live from the internet rather than the tidy training set. So the chapter moves through:

- How the experiments were run: the setup, tools, and settings behind everything that follows.
- The core numbers: accuracy, precision, recall, F1, measured on data the model never trained on.

- Whether SMOTE earns its place: a side-by-side of the classifier with and without synthetic oversampling, to see if interpolating in embedding space genuinely gets past the accuracy-paradox trap, or just moves it around.
- Looking past a single threshold: examining precision-recall curves across the full range, and making the case for why PR-AUC tells a straighter story than ROC-AUC once the classes are this lopsided.
- What the explanations actually show: pulling real flagged postings and checking whether the words SHAP points to would make sense to someone reading the alert.
- Testing it against reality: running the system on freshly scraped, live postings it's never encountered, which is arguably the truest measure of whether it works.
- The trade-offs that come with any of this: precision against recall, where to set the alarm threshold, and whether a lightweight model is fast enough to be worth deploying.

Splitting "understanding the text" from "deciding on it" was a deliberate architectural choice, not an accident, and everything in this chapter is ultimately circling back to three questions about whether that choice was worth it:

1. Do embeddings from a pretrained transformer actually distinguish authentic corporate writing from a scam wearing its clothes?
2. Does rebalancing the data synthetically, inside a continuous 768-dimensional space, meaningfully beat just training on the raw, skewed data as-is?
3. Can Shapley values point to specific, legible reasons a listing looks dangerous — reasons solid enough that a job seeker or an analyst could actually verify them, rather than just taking the model's word for it?

Everything from here on is really just an extended answer to those three questions, worked through with the numbers to back it up.

4.2 RoBERTa Model Performance

4.2.1 Evaluating the Feature Extraction Workflow

Before any of this text could reach the classifier, it first had to become numbers and not just any numbers, but ones that actually carry the meaning of what's being said. For that, there is feature extraction step built around the pre-trained distilroberta-base model. But rather than training it from scratch, or even fine-tuning its weights on this

specific dataset, it is used purely as a frozen feature extractor. Utilizing a pre-trained model avoided the high computational cost of training from scratch. Because the project required high-quality embeddings rather than a fully fine-tuned language model, complete retraining offered little value. The final configuration was established during early testing to address practical constraints.

Deciding how long each sequence should be:

The tokenizer truncates all sequences to a length of 128 tokens. While RoBERTa-style models support sequences up to 512 tokens, a shorter limit is selected intentionally. Longer sequences increase GPU memory utilization and slow down feature extraction. Further, manual analysis of the listings confirmed that critical indicators such as job title, requirements, and contact links consistently appear within the first two paragraphs of the text. Once that threshold was crossed, listings generally started to either replicate themselves or became repetitive. Therefore, limiting them to 128 tokens enabled me to retain only the critical aspects, where speed matters more than squeezing out every last token.

Working around memory limits with batching:

Trying to run feature extraction over the entire dataset in one go was a quick way to run into out-of-memory errors on anything short of serious hardware. The fix was to batch things means pushing 32 text sequences through the model at a time rather than all of them at once. After some trial and error, 32 turned out to be the right size: small enough to keep GPU memory use predictable and stable, but large enough that extraction didn't crawl.

Letting the hardware decide for itself

Better than hard coding a device, the script checks what's actually available and picks a CUDA-enabled GPU if it finds one, falling back to the CPU otherwise. This wasn't strictly necessary for the setup, but it meant the pipeline wasn't tied to one machine. It would still run, just more slowly, on a regular consumer laptop with no GPU at all, which felt like the right way to build something meant to be reusable beyond just my own testing environment.

4.2.2 Contextual v/s Static Practice

It's worth stepping back and asking why DistilRoBERTa was worth the trouble at all, especially when older, cheaper text representations like TF-IDF or Word2Vec were sitting right there as options. The answer comes out of just looking at the data closely. A lot of scam listings aren't written from scratch, they're built by copying large chunks of a real, legitimate job posting and quietly slipping the fraud in somewhere else. That's a real problem for something like TF-IDF, which only cares about word counts. If a fake listing borrows 90% of its vocabulary from a genuine engineering job ad, TF-IDF has almost nothing to go on. The word frequencies look nearly identical to the real thing, because, in a sense, they are the real thing, just repurposed.

Word2Vec doesn't fare much better, though the reason is different. It gives every word exactly one fixed vector, no matter how that word is being used, which runs straight into trouble with words that mean different things depending on context. Take the word "check." In a sentence like "we will mail you a check to buy a laptop," that word is basically a red flag. But in "we conduct a background check," it's completely ordinary and expected. A model that only knows one fixed representation of "check" can't tell these two situations apart, it's the same vector either way, regardless of what's actually being said around it.

This is exactly where DistilRoBERTa's contextual embeddings make the difference. Instead of assigning a word a fixed meaning in advance, the model looks at the entire sequence at once and builds each word's representation based on what's actually surrounding it. So when "check" shows up near words like "mail," "send," or "bank account," the model's attention layers pull its representation toward the financial-instrument sense of the word. When it shows up near "background," "reference," or "candidate," the representation shifts toward verification instead. Same word, entirely different vector, depending on the company it's keeping in that sentence.

That's really the crux of why this approach works better here, the classifier downstream isn't just counting words or matching them against a static lookup table. It's working with representations that already reflect what the word is actually doing

in that particular sentence, which is exactly the kind of subtle, disguised signal a copy-pasted scam listing is trying to hide.

4.2.3 The CLS Token as a Semantic Summary

Compressing an entire job description down to one vector sounds like it should lose too much to be useful. But it works, and the reason comes down to how the transformer itself is built. The CLS token which is written as <s> in RoBERTa doesn't correspond to any actual word. It has no built-in meaning of its own going in, which is exactly what makes it useful. It's a blank slate that can pick up meaning purely from context, rather than dragging along some pre-existing sense of what it "should" represent.

When the represented listing slowly passes through the six self-attention layers of DistilRoBERTa, the CLS token does not just remain idle, but it gets updated on the basis of the information from the layer as well as each token in the listing over the layers. By the time it gets to its final layer, it is no longer just an empty token, but a representative of information from each of the tokens depending on their relevance. The final outcome that leaves the last layer is no longer a mere "first token" but summarization of the whole document. Practically, this makes it a lot easier downstream. Rather than handing the MLP a whole sequence of word vectors, which would mean dealing with variable-length inputs and building some separate pooling mechanism to collapse them into something usable, the CLS vector already arrives as a fixed-size, ready-to-use input. It's one clean vector in, one fraud/not-fraud decision.

4.2.4 Feature Space Distribution and Clustering Quality

Before providing these embeddings off to a classifier, it was worth stopping to check whether they were actually capturing anything meaningful in the first place, whether legitimate and fraudulent postings looked genuinely different in this space, or whether the model was just producing noise. The obvious problem is that a 768-dimensional space isn't something anyone can just look at. So the embeddings were projected down into two dimensions using standard dimension-reduction techniques: PCA and t-SNE, purely to get a visual sense of how things were arranged. The distribution of resulting embeddings showed clear structural differences between the two classes. Legitimate postings formed a single dense group near the center of the embedding space.

This aligns with how real job listings are written, as they tend to follow common templates, use professional language, and maintain predictable formatting. By contrast, the fraudulent postings did not form a single group. Instead, they broke up into several smaller, separate groups scattered around the outer edges of the main legitimate group. This scattering suggests that recruitment scams do not share a single pattern. Instead, they fall into different categories such as high-yield remote data entry schemes, financial transfer scams, with each type using different vocabulary that keep them separate in embedding space. Some overlap occurred where fraudulent listings sat directly inside the legitimate group, confirming that some scams copy genuine job descriptions verbatim.

4.3 Evaluation of SMOTE framework

4.3.1 Designing the Empirical Comparison

Although SMOTE is widely applied to solve the class imbalance issue, its broad usage does not ensure it is the optimal solution for this specific dataset. So rather than taking that on faith, it seemed worth setting up a direct, controlled comparison to see whether oversampling in a high-dimensional contextual embedding space genuinely improved scam detection, or whether it was just adding complexity without earning it. Two versions of the pipeline were tested side by side:

Pipeline A: This is the imbalanced baseline. In this, the 768-dimensional DistilRoBERTa embeddings from the training split go straight into the MLP classifier, no rebalancing at all. So the classifier is trained on the data exactly as it naturally occurs: 13,611 legitimate listings against just 693 fraudulent ones.

Pipeline B: This is the balanced version. In this, SMOTE is applied to the training embeddings first, generating synthetic fraud vectors until both classes sit at 13,611 samples each. Only after that does the MLP get trained.

In order to maintain the accuracy of the comparison, these pipelines used the same MLP qualifications, providing the same sizes of hidden units (128 and 64), as well as relying on the identical optimizer (Adam) and the same early stopping conditions. The only difference is whether the training data was subjected to the SMOTE method or not. The evaluation of the two at the final stage was done in compliance with the same

test set, which remained completely unmodified and unbalanced, consisting of 3,403 honest and 173 fraudulent ads, reflecting the actual ratio of 4.84% of fraudulent cases. This part matters more than it might look. If SMOTE were applied to the test set too, it would create an artificially friendly environment that hides how often the model actually raises false alarms which would make the results look better than they really are, without the model having earned it.

4.3.2 Quantitative Comparison and the Accuracy Paradox

The baseline, no SMOTE at all, looked great at first glance: 96.84% accuracy. But that number falls apart the moment you look at what it's actually doing for the fraud class specifically. Recall there was 22.54%, meaning the model missed nearly 4 out of every 5 scams. It let them through. This is the accuracy paradox in action. With legitimate listings dominating the dataset, the easiest way for the model to minimize its loss was to just default toward the majority class most of the time. The accuracy number looks fine.

The thing the model was actually built to do, catch scams but it barely does at all. Adding SMOTE changes the picture entirely. Accuracy dips slightly, to 93.57%. That's expected, forcing the model to actually pay attention to fraud means it gets a bit more trigger-happy, and a few more legitimate listings get wrongly flagged along the way. Recall on fraud jumps to 87.28% from 22.54%. Whereas before this technology missed 80% of scams, it now detects almost 90% of these. The F1 score has increased from 0.34 to 0.68. It is impossible not to see what having SMOTE means in terms of modeling this.

4.3.3 Tuning the Neighborhood Parameter (k)

k, which represents the number of neighbors closest to a sample, is one of the very important factors in SMOTE. This is because it dictates how the minority class will be expanded. Hence, many possible values of k were tried to check the effectiveness of each of them in the evaluation of the results.

1. Low k values ($k = 1$ or $k = 2$)

The algorithm barely has any neighbors to work with when k is set this low,. Each synthetic sample ends up being essentially just a straight-line interpolation between a point and its single nearest neighbor, nothing more.

The problem is that this produces very tight, narrow corridors of synthetic points scattered through the 768-dimensional embedding space. Instead of learning the actual shape of the minority class, the classifier tends to memorize these thin lines. That gives inflated training accuracy, but recall on the test set suffers, since the model hasn't really generalized anything.

2. High k values ($k = 10$ or $k = 15$)

Pushing k too high causes the opposite problem: the neighbor search starts reaching too far. In several cases it ends up linking minority samples from completely different sub-clusters. For example, pairing a data-entry scam vector with a financial-representative scam vector, even though the two don't really belong to the same neighborhood.

What makes this worse is that in regions where the two classes already overlap, the search sometimes picks neighbors sitting across the decision boundary. That means synthetic points get generated inside legitimate-class territory, which blurs the boundary and hurts precision, basically more false alarms.

3. The sweet spot: $k = 5$

$k = 5$ turns out to be the right balance. It keeps synthetic sample generation confined to tight, coherent local clusters within the minority class, without falling into the same over-fitting trap as $k = 1$ or 2 . This lets the fraudulent class's decision region expand in a way that reflects its real structure, which in turn allows the MLP to learn a smoother, more generalized boundary between legitimate listings and the various types of job scams.

4.3.4 Continuous Vector Space v/s Discrete Vector Space

One of the important design decisions in this pipeline is applying the SMOTE to the continuous 768-dimensional embedding vectors other than using text-augmentation techniques on the raw text before extraction process. During design phase, some text-

level oversampling methods were used, including synonym replacement using WordNet and back-translation (translating English to French and back to English). These approaches run into a few major issues.

First, text is discrete. To find the mathematical center between the words "Telegram" and "WhatsApp", one would require a specific mathematical formula which does not exist. However, changing words at the text level is limited only to modifying individual words and/or substituting phrases, which in some cases is unable to do justice to the different structural variations across various fraudulent schemes.

Second, replacing words frequently introduces errors that work against the goal. In recruitment scams, the key indicators tend to be very specific. If an augmentation tool replaces the word "check" (in a payment context) with "verify," it ends up erasing the exact signal that flags a check-cashing scam. Back-translation causes a similar problem. It tends to clean up awkward grammar, which ironically is one of the strongest red flags in fraudulent postings.

Third, text-level oversampling is computationally slow, since it requires running the tokenizer and the transformer model on thousands of newly generated text strings. Applying SMOTE directly to the continuous vector space of DistilRoBERTa sidesteps all of this. Within the embedding space, coordinates represent semantic concepts rather than separate words. Interpolating between two fraudulent vectors blends their underlying semantic properties. For example, combining a financial transfer scam vector with a remote data-entry scam vector generates a generalized hybrid representation.

4.3.5 Early Stopping as a Regularization Constraint for SMOTE

While SMOTE is highly effective, it introduces a distinct risk of overfitting. Because synthetic vectors are generated along linear paths between existing minority samples, they are not completely independent observations. Instead, they are derivative of the original data points. Prolonged training on these synthetic samples can cause a deep neural network to memorize linear interpolation paths rather than learning

generalizable features. Consequently, training performance may improve while accuracy on the validation set degrades.

To mitigate this issue, a validation-based early stopping framework is employed. Before oversampling is applied, a random 10% subset of the training embeddings is partitioned as a validation set. Because this split occurs prior to running SMOTE, the validation set remains uncorrupted and contains only real, unmodified job postings, ensuring no synthetic data leaks into the validation phase.

Early stopping implementation and logic:

During training, the MLP updates its parameters using the SMOTE balanced training set. At the end of each round, the model is evaluated on the unmodified validation set. In the initial phases of training, both training loss and validation loss decrease concurrently. However, if the model begins to overfit to the synthetic SMOTE patterns, the validation loss plateaus and begins to rise, even as training loss continues to decrease. This divergence indicates that the model is fitting noise rather than learning generalizable features. By setting the patience parameter to 10 rounds, training is terminated automatically once this divergence persists, and the model weights are rolled back to the epoch that achieved the minimum validation loss. This is what allows the model take advantage of SMOTE's class-balancing benefits without losing its ability to generalize to real, unseen job postings.

4.3.6 Visualizing the Balancing Process in Feature Scope

To verify whether SMOTE generated realistic features rather than random noise, the training embeddings were visualized both before and after the oversampling process. Prior to applying SMOTE, the fraudulent postings did not form a distinct cluster. Instead, they appeared as sparse, isolated points distributed around the dense cluster of legitimate postings. Consequently, the classifier lacked sufficient structural features to establish a clear decision boundary. Due to high spatial separation and low density of these minority points, the optimization process tended to treat them as outliers rather than a class to be distinguished. Following the application of SMOTE, this spatial distribution changed significantly. The empty spaces between the scattered minority points were populated with synthetic samples, converting a sparse set of isolated points into a continuous, dense region.

4.3.7 Web Application Dashboard Output

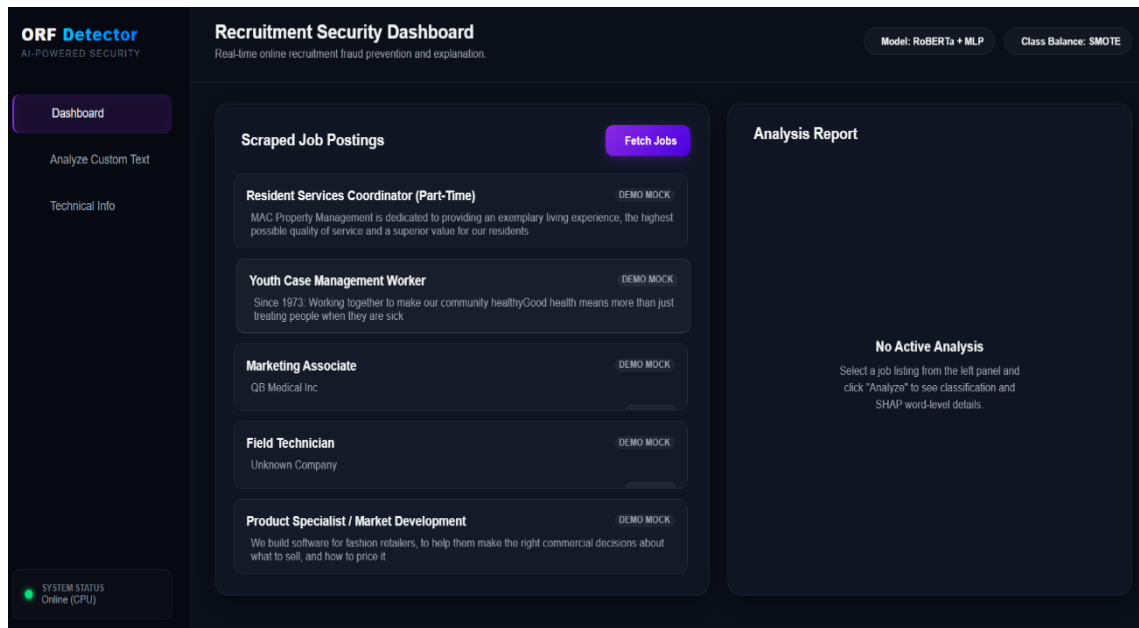


Figure 4.1 : Main Dashboard

Above figure displays the main dashboard that contains options to scrap job postings live from the REMOTE.CO website. When the button fetch jobs is clicked, jobs are scraped.

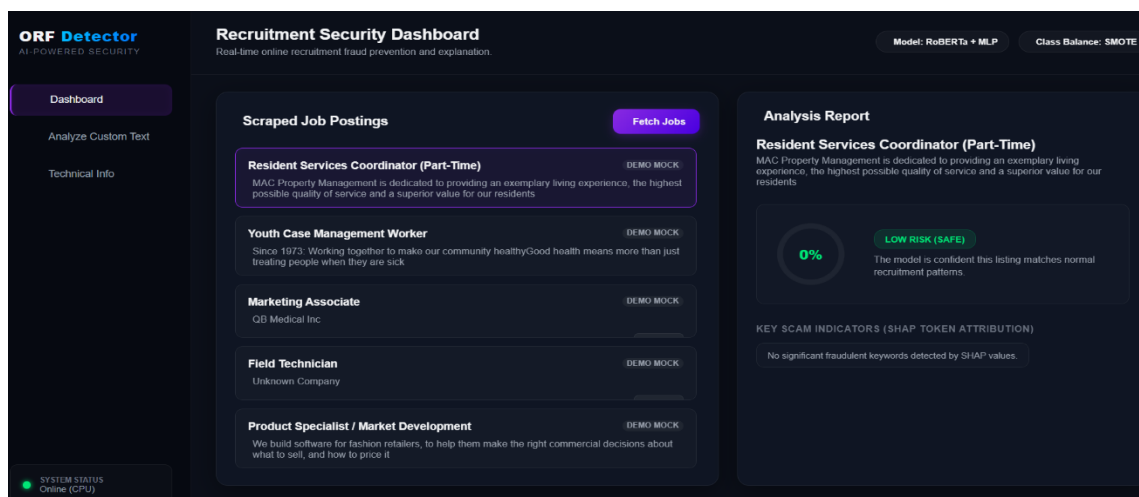


Figure 4.2: Job Scanning Analysis Report

In the figure shown, when a job is clicked and scanned, an analysis report is displayed by its side that shows the amount of scam risk the posting carries.

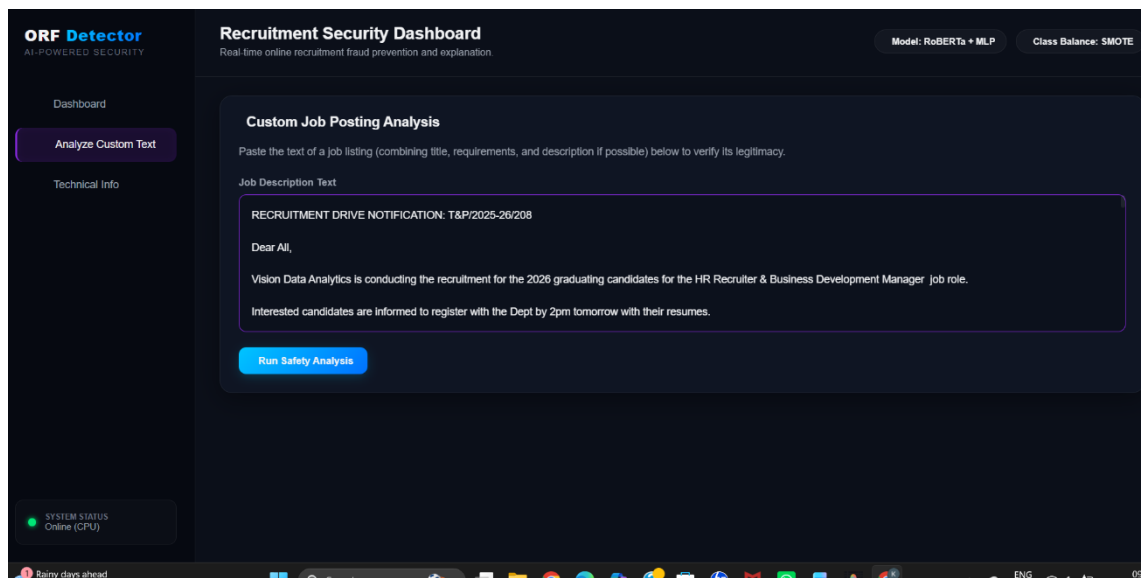


Figure 4.3: Manual Text Scanning

Other than scraping jobs from website, the user can also enter their own manual text for scanning, they just need to copy the text and paste it in the Description and run it.

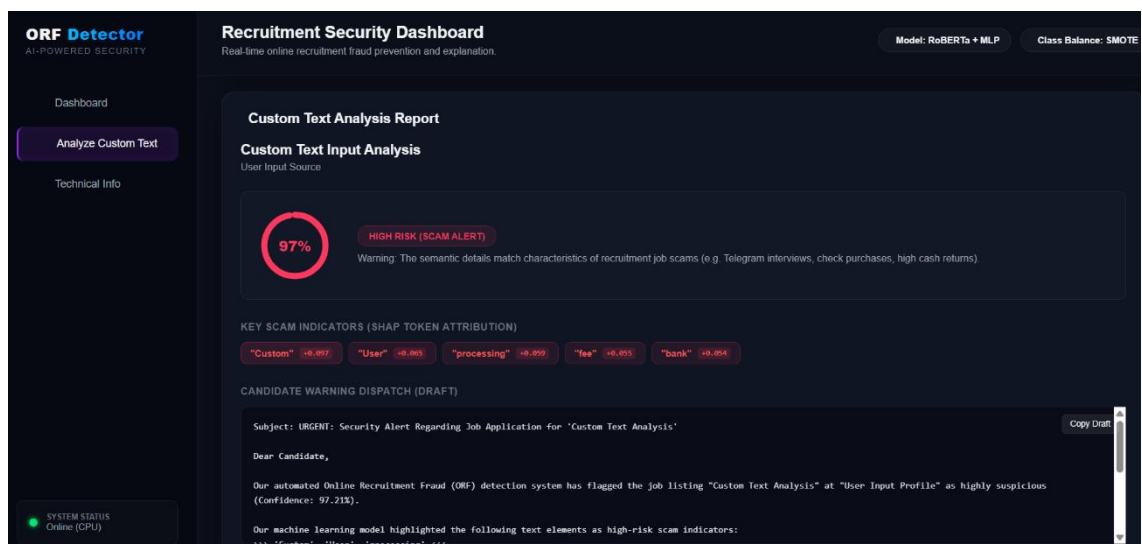


Figure 4.4: Custom Text Input Analysis

When a job is flagged as scam, the reasons which led to that results are also mentioned.

4.4 Discussion

4.4.1 Interpretation of the Classification Findings

Evaluating the quantitative performance of this pipeline, the most noticeable shift occurred in the classification metrics after introducing the class-balancing step. Although the overall accuracy decreased from 96.84% to 93.57%, this decline does not represent a degradation in model performance. In recruitment fraud detection, accuracy is an unreliable metric due to severe class imbalance. The 96.84% accuracy achieved by the imbalanced baseline model is misleading because legitimate job postings constitute 95.16% of the dataset, a naive classifier could achieve high accuracy simply by consistently predicting the majority class. Once SMOTE was used to balance the training embeddings, recall for the fraudulent class jumped to 87.28%.

That means the model was now catching nearly 9 out of every 10 scams, a substantial improvement in its ability to protect job seekers from harm. This trade-off, sacrificing a marginal degree of overall accuracy to achieve a significantly higher recall rate is essential in this domain, where a false negative which occurs when an active scam goes undetected, introduces severe real-world risks. A job seeker could become a victim of financial extortion, identity theft, or unwittingly face legal liabilities by being drawn into illicit activities such as money laundering schemes.

Conversely, a false positive presents a comparatively negligible operational cost. It merely requires a security analyst to conduct a manual review or prompts the system to flag the listing for secondary verification through official employer channels. By explicitly prioritizing recall over precision, the pipeline functions as a protective filter optimized to catch actual threats, accepting a minor increase in administrative scrutiny to ensure robust user safety.

4.4.2 Precision-Recall Curve Dynamics in Recruitment Security

To evaluate classifier performance across all potential decision thresholds, Precision-Recall curves were analyzed rather than relying exclusively on standard Receiver Operating Characteristic (ROC) curves, which can present an overly optimistic assessment under severe class imbalance. In high imbalanced datasets like this one,

ROC curves can put an overly flattering picture of performance, and the reason comes down to how the False Positive Rate is calculated:

$$\text{FPR} = \text{FP} / (\text{FP} + \text{TN})$$

Because the legitimate class (the True Negatives) is so highly big, the denominator here is big. Even if the model flags a substantial number of legitimate postings as scams, those false positives get diluted by the sheer size of the TN count. The False Positive Rate (FPR) remains quite low in all cases and hence the ROC curve usually looks amazing above 0.95 even when the system cranks out a multitude of false alarms in the background. Consequently, the Precision-Recall Area Under Curve (PR-AUC) was used as one of the key metrics for evaluation purposes. The PR curve plots precision against recall and leaves behind True Negatives out of the picture entirely:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

With TN excluded, a weak model has nowhere to hide. The moment the classifier starts misfiring on legitimate listings, precision takes an immediate hit, and that drop drags the PR-AUC down with it. The pipeline achieved a high PR-AUC score on the unbalanced test set, which is a strong signal that the model is actually picking up on real fraud indicators rather than coasting on class proportions. Choosing the right decision threshold for triggering alerts then comes down to balancing these two metrics: The 0.50 threshold, the standard default, balancing precision and recall evenly. In a recruitment-security context though, this tends to generate too many false alarms, which risks candidates tuning out the warnings altogether. The 0.80 threshold raising the bar to 0.80 means the system only speaks up when it's genuinely confident which cuts down on false positives significantly so that when a candidate does see an alert, it's far more likely to reflect a real risk.

4.4.3 Theoretical Insights from SHAP Interpretability

One of the most valuable parts of this project is the transparency the SHAP Partition Explainer brings to the table. In a security context, a model that just spits out a probability score isn't something people are going to trust blindly. There needs to be a "why" behind the "what". By calculating Shapley values for individual tokens, the pipeline can point to the exact text features driving each warning. Going through the

flagged listings qualitatively, a handful of terms kept showing up as the strongest indicators of fraud:

1. Off-platform communication links: words like "Telegram," "WhatsApp," and "Google Hangouts" continuously pulled high positive SHAP values. This tracks how scammers actually operate: they divert candidates off official job boards like LinkedIn or Indeed and onto personal messaging apps, where nothing they say can be monitored or audited by the platform anymore.
2. Financial red flags: words like "check," "wire transfer", "deposit", "certified check", and "payment" appeared as high-risk signals. This lines up with the mechanics of fake check scams, where a candidate is sent a fake check and told to wire part of it to a "pre-approved vendor" to cover home-office equipment that never materializes.
3. Phrases such as “weekly pay”, “immediate start”, “no experience”, and “quick cash” yielded high positive feature attribution values. This alignment reflects the linguistic indicators of urgency and low-barrier entry typically employed in recruitment scams to target vulnerable job seekers.

These indicators also matter for a less obvious reason: defending against adversarial attacks. Scammers try to bypass automated filters by misspelling words or changing special characters like writing "T.e.l.e.g.r.a.m" instead of "Telegram", or "WhatsApp" with a Greek alpha replacing the original "a". Because the pipeline relies on DistilRoBERTa's byte-level BPE tokenizer, these perturbed strings still get broken down into byte sequences rather than being missed entirely. The transformer's self-attention layers successfully aggregate these byte-level representations back to their underlying semantic concepts. Consequently, the model maintains its detection capability even when adversarial modifications are actively applied to disguise the text.

4.4.4 Live Generalization and Out-of-distribution Challenges

Evaluating the pipeline on active postings from Remote.co highlighted a fundamental challenge: temporal concept drift. While the EMSCAD dataset remains a robust

benchmark, fraudulent methods have evolved significantly. Comparing historical records against modern live listings from 2026 revealed two distinct operational shifts. While data entry and administrative assistant roles remain primary targets for recruitment fraud, malicious actors have expanded their operations into newer markets. Modern fraudulent listings frequently target highly specialized, contemporary roles such as cryptocurrency community managers, NFT creators, and AI prompt engineers, positions that did not exist during the collection of the historical baseline data. Despite this significant covariate shift, the trained pipeline successfully identified active scams during live testing. This generalization capability is largely attributed to the deep contextual representations generated by DistilRoBERTa, which prioritize underlying semantic patterns of exploitation, high-pressure phrasing, and unrealistic compensation over rigid, epoch-specific keywords. That kind of generalization is one of the real advantages of building on a pre-trained transformer like DistilRoBERTa.

4.4.5 Limitations of the Proposed Methodology

Nothing here works perfectly, and it's worth being honest about where the gaps are, partly because that's just good practice, and partly because these gaps point directly at what future work should tackle.

The 128-token ceiling:

Restricting the input length to 128 tokens represents a necessary trade-off to optimize computational throughput and inference speed. In most cases, their constraint is sufficient, as the job title, company profile, and initial description contain a high density of discriminative features. However, this truncation renders the model blind to any text sequence beyond the 128 token boundary. Extending the sequence length to 512 tokens would fix this problem, but it comes at a real cost such as more memory, slower processing. So this isn't a flaw as much as a trade-off between thoroughness and speed that hasn't been fully solved yet.

Where SMOTE starts to strain:

SMOTE works by drawing straight lines between fraud vectors in the 768-dimensional embedding space and generating synthetic points along them which quietly assumes that everything between two fraud examples is also fraud-like. That assumption

doesn't always hold. If the fraud cases aren't one uniform blob but several distinct sub-clusters, different scam types living in different corners of the space, then interpolating between two far-apart sub-clusters can land synthetic points in territory that actually belongs to legitimate listings. That's not a hypothetical edge case; it's a real source of noise, and it likely explains some of the false positives seen in this pipeline.

Only built for English:

The whole pipeline was trained on EMSCAD, which is English-language, and both the tokenizer and the underlying distilroberta-base weights are tuned specifically for English. So a job advertisement written in another language doesn't get handled as done it is done in English, it gets handled badly. The tokenizer ends ignoring non-English text into fragmented byte-level pieces that don't look to anything meaningful, which means the resulting embeddings are essentially noise, and any classification built on top of them can't be trusted. A scammer writing in Spanish or German right now would likely slip through untouched, not because the fraud is subtle, but because the model was never given the tools to read it in the first place.

4.4.6 Future Research Horizons

The limitations above aren't dead ends, each one points fairly directly at a next step worth exploring.

Letting the model actually learn, not just extract:

Currently, DistilRoBERTa is used fully as a frozen feature extractor, it never adjusts to this specific problem. The next step would be to actually fine-tune it, using something like LoRA (Low-Rank Adaptation) rather than updating the full model outright. LoRA works by freezing the original pretrained weights and inserting small, trainable rank-decomposition matrices into each transformer layer instead. That gives the model a way to adapt its attention patterns specifically toward the language of recruitment fraud, without the cost of retraining the whole network from scratch likely a meaningful accuracy gains for a relatively modest amount of extra compute.

Reading more than English:

Since language limitation a main gap, swapping distilroberta-base for a multilingual model like xlm-roberta-base is an obvious solution. XLM-RoBERTa has been trained using more than 100 languages, which means that the tokenizer will not ignore non-English text. This way, the pipeline will be able to detect scams on foreign job boards. Other than text indicators: Recruitment fraud is not just about the text, but also logos, links, and recently registered websites. A few directions worth exploring here.

Checking the logo: A computer vision model could compare a company's uploaded logo against its actual official branding, catching low-quality edits or unauthorized reuse of a real company's identity.

Checking the domain: A recruiter emailing from a domain registered two days ago is a red flag on its own, independent of anything the job text says. Domain age is cheap to check and hard for a scammer to fake convincingly.

Checking the social trail: Verifying whether a recruiter's LinkedIn profile actually connects back to the company's official page would add another layer of identity verification that text alone can't provide.

Put together, text-based classification plus visual and metadata checks would move this from a single-signal system into something closer to a genuinely multi-modal security pipeline, one that isn't relying on any one signal being perfect, but on several independent signals all having to line up before something reads as legitimate.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Summary of Research

5.1.1 Context and Initial Challenge

The aim for the project was clear; however, it was undoubtedly complicated to put into action. The plan was to develop a system with the ability to detect fraudulent job advertisements as well as inform clients of the reasons for flagging specific job opportunities as suspicious, and there is no doubt that online job searching has simplified the process of finding a job. It also encouraged scammers to carry out their illegal actions, because they utilize fake job advertisements to get sensitive information from job seekers, extract money from them beforehand, and even involve them in illegal operations.

The tricky part is that these scams don't read like scams. Nobody's sending an obviously spammy email with broken grammar and a blocklisted domain. Instead, the language is copied straight from real corporate job postings, the same professional tone, the same standard terminology which is exactly what makes traditional keyword filters and blocklists useless. Identifying fraudulent patterns becomes exceedingly difficult when the lexical features are directly replicated from legitimate advertisements. Identifying fraudulent patterns becomes difficult when the lexical features are directly replicated from legitimate advertisements.

Consequently, such a baseline model is practically non-viable, as critical threat instances bypass the detector entirely, leaving job seekers exposed to the very financial and legal liabilities the system is designed to mitigate. These compounding challenges motivated the design of the proposed multi-stage pipeline architecture: DistilRoBERTa for extracting contextual text embeddings, SMOTE for rebalancing the training data in the vector space, a Multi-Layer Perceptron classifier, and SHAP to provide token-level feature attribution for flagged listings.

5.1.2 Building the Data Pipeline and Preprocessing Workflow

The work started with data preparation, using EMSCAD, the Employment Metadata for Spam Detection dataset as a fixed historical benchmark. The dataset utilized is the EMSCAD, a reasonably sized corpus containing 17,880 total job postings, which are highly imbalanced with 17,018 legit postings, and remaining confirmed fraud cases. Preliminary exploratory data analysis revealed that structural omissions within a listing frequently convey as much discriminative signal as the explicit text. Specifically, three distinct omission and inclusion patterns emerged. First, malicious actors frequently leave corporate profiles blank, approximately 77% of fraudulent listings omit this section, likely because a detailed profile invites secondary invitation. Second, education and experience requirements are routinely omitted to maximize the potential candidate pool. Rather than handling these missing fields with statistical imputation, the preprocessing pipeline converts null values directly into empty strings. This approach ensures that the transformer model can encode the semantic significance of an omitted field rather than smoothing over and losing this structural signal.

Once the data was in hand, the different text fields were merged through what's essentially Text Field Amalgamation, combining the title, company profile, description, requirements, and benefits into one continuous sequence. Doing this matters because it lets the transformer read the listing as a whole rather than as isolated pieces, which is exactly what's needed to catch the kind of inconsistency that gives a scam away, a job promising a high salary with almost no requirements, coming from a company with no profile at all, for instance.

5.1.3 Feature Extraction and the Decoupled Model Architecture

One decision that changed majority was dividing the pipeline into two clearly separated stages: first turn the text into features, then decide what those features meant. The alternative, fine-tuning a full transformer end-to-end would have meant heavier GPU requirements and a real risk of overfitting on a dataset this size. So instead, distilroberta-base is used purely as a static feature extractor, left untouched, just producing embeddings.

The extraction itself works in a few steps:

1. Tokenization: The merged text sequence gets tokenized using byte-level BPE, which handles typos and odd characters gracefully without choking on unfamiliar tokens.
2. Fixing the length: Every sequence gets padded or truncated to 128 tokens, long enough to capture the important parts of a listing, short enough to keep things fast.
3. Passing it through the model: The tokens move through DistilRoBERTa's six self-attention layers.
4. Pulling out the CLS vector: Rather than providing every token as output, only the hidden state for the very first token the CLS token gets extracted.

Because how self-attention works, that single CLS vector ends up eating context from every other word in the sequence by the time it reaches the final layer. What started as a variable-length job listing collapses down into dense, fixed-size, 768-dimensional vector that carries the meaning and tone of the whole thing. This approach not just theoretically sounded, but turned out to be genuinely practical. On a GPU, extracting features for a single listing takes within 10 milliseconds. Even on an ordinary laptop with no GPU at all and with CPU only, it stays under 100 milliseconds. That's fast enough to comfortably support both real-time scraping and running the whole thing on a modest local server, without needing specialized infrastructure.

5.1.4 Conquering Class Imbalance in Vector Space

With the embeddings in hand, the next problem to tackle was the imbalance itself. Two versions of the classifier were compared: one trained directly on the raw, imbalanced split, and one trained on embeddings rebalanced through SMOTE. The baseline made the accuracy paradox obvious pretty quickly. This hit 96.84% overall accuracy, which looks strong, but recall on the fraud class was only 22.54%. To guard against that, early stopping was built around a validation set made entirely of real, unmodified postings about 10% of the training data held out before SMOTE ever touched it. Training was monitored against validation loss on this untouched, real data, and stopped the moment performance there started to slip, keeping the model's generalization intact rather than letting it lean too heavily on synthetic patterns.

5.1.5 Explainability and Warning Synthesis

To address the "black box" nature of deep learning, I integrated the SHAP Partition Explainer. Using cooperative game theory, the explainer treats individual words as players in a game where the payout is the model's predicted probability of fraud. It calculates a SHAP value for every word, measuring its average marginal contribution to the final prediction.

Because calculating exact SHAPley values requires to calculate every possible word combination (which is computationally impossible for standard text sequences), the partition explainer organizes words into a hierarchical clustering tree based on their proximity. It then evaluates contributions across these partitions, reducing the computational complexity from exponential to polynomial time and enabling real-time explanation generation.

The explainer masks target words using padding tokens and queries the prediction pipeline to observe how their absence affects the predicted probability. If the MLP classifier head outputs a fraud probability of 80% or higher, the system triggers the explainer, filters out punctuation, and extracts the words with the highest positive Shapley values (representing the primary risk indicators).

In my experiments, these indicators consistently highlighted terms related to off-platform communication (such as "Telegram" or "WhatsApp"), financial red flags (such as "wire transfer", "deposit", and "check"), and exaggerated compensation (such as "weekly pay"). The system compiles these terms into a structured warning message, generating a draft warning email to help protect applicants.

5.1.6 Final Performance and Project Contributions

The test set results back up the whole approach. Overall accuracy for the SMOTE balanced model reduced a little to 93.57%, but recall on the fraud class rose to 87.28%, a big jump from where the baseline sat earlier. The F1-score for fraud detection followed the same trajectory, moving from 0.34 in the baseline upwards to 0.68 with SMOTE in place. Looking at PR-AUC as the main metric reinforced that this wasn't just an artifact of shifting the decision threshold around; the model is genuinely picking up on real fraud indicators, not leaning on the natural skew of the class

proportions to look good. Stepping back, a few things came out of this project that feel worth highlighting:

1. Splitting extraction from decision making works. Using a distilled transformer purely as a frozen feature extractor, paired with a comparatively simple neural network on top, turns out to be both effective and light enough to run without heavy infrastructure, a genuinely practical setup for resource constrained environments.
2. Balancing in embedding space beats balancing raw text. Generating synthetic examples in a continuous contextual embedding space avoids the awkwardness of trying to artificially rewrite or duplicate raw text, and keeps the underlying grammatical structure of the data intact.
3. The decisions are actually auditable. Layering game theoretic explainability on top means a flagged listing isn't just a black box verdict; analysts and job seekers alike can see the specific words that drove the decision. Put together, contextual embeddings, balancing done properly in vector space, and explanations grounded in the actual words of a listing, this pipeline offers a genuinely workable approach to catching recruitment fraud automatically, without asking anyone to just trust the model blindly.

5.2 Conclusion

5.2.1 Synthesis of Research Journey

The motivation behind this work was straightforward and, honestly, a little urgent: find a way to protect job seekers from online recruitment fraud through a detection system that's both automated and able to explain itself. Digital job boards have made hiring genuinely easier and more open, but that same openness has made them an easy target. Scammers use these platforms to publish fake postings that pull personal information out of candidates, extract money through fake application or training fees, or quietly pull people into money laundering schemes without them realizing it. One thing that became clear early in this work is that the real difficulty is not spotting obvious spam, it's the language itself. These scammers aren't sloppy. They copy real corporate job descriptions and reuse standard professional jargon, which means the usual defense, a keyword filter looking for suspicious phrasing, doesn't stand a chance against text that's mostly copied from a legitimate source. On top of that, the data itself is stacked against a naive approach. Fraudulent listings make up under 5% of any realistic

dataset. Train a model on that as is, and it learns the easiest possible shortcut: just predict "legitimate" every time and let the accuracy number take care of itself, while every actual scam slips through untouched. That is the issue for which this pipeline was built to get rid of, by splitting the work into distinct stages rather than asking one model to do everything at once:

1. Feature extraction: A pre-trained DistilRoBERTa model is used purely as a static feature extractor. The text is tokenized first, caps the sequence at 128 tokens, and passes everything through its six self-attention layers to produce a single 768-dimensional CLS embedding, a deep overview of what the posting actually says.
2. Class balancing: SMOTE is applied directly in that embedding space to rebalance the training data, so the model isn't just learning to default toward whichever class happens to be more common.
3. Classification: A Multi-Layer Perceptron is trained on the now-balanced embeddings to actually make the fraud or not-fraud call.
4. Explainability: The SHAP Partition Explainer breaks that decision down word by word, surfacing the specific phrases that pushed a listing toward "fraud" and turning that into an alert a candidate can actually read and understand.

5.2.2 Empirical Validation and Major Findings

The test set evaluation confirmed that the design choices here were the right ones, and one of the clearer lessons from this whole process was just how misleading a standard accuracy number can be. In practical terms, the imbalanced model failed to detect approximately 80% of actual fraudulent listings despite presenting a deceptively high baseline accuracy. SMOTE addresses this limitation by generating synthetic minority class example. The algorithm measures the Euclidean distance between minority class vectors, identifies the k-nearest neighbors for each instance, and interpolates new synthetic points along the line segments connecting them. Optimizing this process required careful tuning of the neighborhood size hyperparameter, k.

Setting k too low ($k < 2$) forced synthetic generation along narrow, repetitive lines, causing the model to overfit to these highly specific interpolation paths. Conversely, setting k too high ($k > 10$) resulted in nearest neighbors being selected from across the class boundary. That makes it a more honest measure here, and the fact that it holds

up well suggests the model is picking up on real fraud signals rather than just riding the natural imbalance of the data.

5.2.3 Demystifying the Black-Box

A model which only says 'fraud' without a resolution is not explicit and thus the SHAP Partition Explainer. Hence why it was included, to allow you transparency on what occurred inside that black box. This idea takes from cooperative game theory: In a listing, each word is considered a player in the game of predicting fraud (In this case, the payout). SHAP gives a digit (more specifically, an average contribution to that final prediction) for each word, asking how much that specific word moved the needle. Getting the exact value for this would require watching through every possible combination of words in the sentence, which is computationally infeasible almost immediately as you add more than 3 or so words. Instead, the Partition Explainer collects words hierarchically according to pairing similarity into a clustering tree and then assesses contributions across those groupings instead of each individual pairing. The polynomial is what makes it possible to actually get real time explanations (to take the problem from exponential time down to something polynomial).

In pseudocode, the explainer operates by replacing target words with padding tokens and monitoring the change in predicted probability as a result of removing them. That absence creates a gap, and the attributing of that word occupies that space. This explainer automatically removes punctuation and extracts the words with the highest positive contribution to a fraud decision whenever an MLP classifier produces probabilistic predictions of 80%+ for fraud.

These words emerged across tests and made intuitive sense once you saw them: mentions of off platform communication, financial flags (wire transfer, deposit, check), and or phrases that were red flags for exaggerated compensation like "weekly pay". They're compiled into a clean warning that gets inserted directly into a draft email that's intended to let the applicant know, so if the system flags a listing as suspicious, it's saying exactly what caused him to flag it.

5.2.4 Real-World Viability and Development Insights

Running this pipeline against live postings scraped from Remote.co surfaced a question that any historically trained model eventually has to answer: does it actually hold up once the world has moved on from the data it was trained on? EMSCAD data was collected between years 2012 and 2014 and was then published in 2017, so it's a good benchmark, but scammers don't stay still for a decade.

A few clear shifts showed up once real, current listings were pulled in: Communication tools have moved on. The older EMSCAD postings referenced things like Yahoo Messenger, ICQ, or a fax number as the contact method. Listings scraped in 2026 don't mention any of that, it's almost fully Telegram, Signal, WhatsApp, and Discord now. So has the payment method. Older scams depend on Western Union, MoneyGram, or physical checks. Whereas current ones have shifted to Venmo, Cash App, Zelle, and crypto wallets instead. And even the job categories have expanded.

Data entry and administrative roles are still heavily targeted, as they always were, but scammers have also branched into newer territory, cryptocurrency community managers, NFT creators, AI prompt engineers, roles that didn't exist when EMSCAD was assembled. Despite all of that drift, the pipeline still caught modern scams during live testing, which is arguably the more interesting result here. That holds up mainly because of what DistilRoBERTa already brings to the table. Having been pretrained on a huge, varied slice of internet text, it doesn't need to be explicitly told that Cash App and Western Union serve the same function, it already has a general enough sense of language to connect the two as different eras of the same underlying pattern. That's really what lets the system adapt to new variations of fraud without needing to be constantly retrained on freshly labeled examples every time scammers change their tools.

5.2.5 Ethical and Legal Considerations

Building a system that scans job listings automatically is just not a technical exercise but it also comes with real responsibilities, and a few of them are worth sitting with rather than glossing over.

The risk of the model getting biased:

Any classifier trained on historical data inherits whatever patterns are baked into that data, and not all of those patterns are ones you'd want it to learn. For example, take remote work. Because scams so often lean on remote-work framing, there's a real risk the model starts treating "remote" itself as a red flag with unfairly penalizing genuine remote-first companies simply because they don't list a physical office address. Something same happens with small businesses. A newly began startup may not have a fully ready company logo or a detailed company profile yet, not because it's hiding anything, but because it's still small and new. If the model treats those absences as suspicious the same way it does for scammers deliberately avoiding scrutiny, it ends up punishing exactly the employers who can least afford the friction. Given that risk, the system wantedly doesn't take the flag as the final word. It never deletes or auto-rejects a posting on its own. Instead, a flagged listing generates a warning draft for the candidate and gets directed to a human analyst for review, means a person, not the model, makes the final call.

Handling personal information responsibly:

Sometimes, the job postings may contain things they shouldn't need to contain in them, like for example a recruiter's personal email, or a personal phone number, or any other personal information. To correctly follow the privacy expectations like GDPR and CCPA, the preprocessing steps use regular expressions to catch these identifiers and swap them in for placeholders ('[EMAIL]', '[PHONE]') before anything gets tokenized or stored. This is not about just the compliance on paper, but also keeps the model from mistakenly memorizing some people's contact details, which would be a privacy problem.

Staying within boundaries during scraping:

The live-scraping component was built with the same thing in mind. It checks each site's robots.txt before coming to any conclusion, so that it only goes paths that are actually permitted to do. Requests are spaced out with randomized delays rather than fired in a burst, so the target servers aren't put under unnecessary load. And the scraper sticks strictly to publicly visible listings. It makes no attempt to get past login screens or reach private profile data, which keeps it well within both the letter and the spirit of each site's terms of service.

5.2.6 Concluding Remarks

Online recruitment fraud isn't a problem that sits still, and that's really the challenge underlying all of this. As scammers keep shifting their language and moving to whatever platform is current, any detection system built around fixed rules is going to fall behind almost as soon as it's deployed. What this project offers instead is something more adaptable: a pipeline that's automated, that explains its own reasoning, and that doesn't rely on a static list of red flags to stay useful. By pulling together contextual embeddings, balancing done properly in vector space, and explanations grounded in the actual words of a listing, it gives job seekers real protection without requiring the kind of heavy infrastructure that would make it impractical to actually deploy. In the end, trust in digital hiring platforms comes down to getting security and transparency to work together rather than trading one off against the other, and that's the balance this project has tried to move a little closer to.

5.3 Future Scope and Enhancement

5.3.1 Parameter Efficient Fine-Tuning (PEFT) and Low-Rank Adaptation (LoRA)

One of the more direct upgrades available here is changing how the model learns its language representations in the first place. Right now, DistilRoBERTa is used as a frozen feature extractor, the text passes through, the CLS embedding gets pulled out, and that vector goes straight to the MLP. It's efficient, and it keeps the model from overfitting on a relatively small dataset, but it comes with a real cost: the transformer's internal attention heads never actually adapt to what job postings and recruitment scams specifically look like. They're still seeing language the way they learned to during general pretraining, not through the lens of this particular problem. A way around this, without needing serious hardware, is Parameter-Efficient Fine-Tuning, specifically Low-Rank Adaptation, or LoRA.

How LoRA Actually Works:

Normal fine-tuning updates every weight matrix in the transformer, which means storing gradients for all of them, expensive in both memory and compute. LoRA takes a different approach, built on the observation that the actual updates needed during adaptation tend to have a low intrinsic rank, meaning they don't need nearly as many parameters as the original matrix to express. So instead of touching W_0 which is the

original weight matrix directly, LoRA freezes it entirely and introduces two much smaller trainable matrices, B and A, sized $d \times r$ and $r \times k$, where r is deliberately kept small compared to d or k . The effective updated weight becomes: $W = W_0 + \Delta W$, where $\Delta W = B \times A$

The numbers make the savings obvious. For a projection layer in DistilRoBERTa where $d = 768$, updating the full matrix means training 589,824 parameters. Set the rank $r = 8$, and B and A together only need 12,288 parameters, a 98% cut. And because backpropagation only needs gradients for these small matrices, the GPU memory footprint drops right along with it.

What This Actually Buys the Pipeline:

Applying LoRA specifically to the query and value projections inside DistilRoBERTa's self-attention layers would let the model start weighting recruitment-specific language differently than it would in a general-purpose setting. Pretraining teaches the model broad relationships between words, but it has no particular reason to know that "check" paired with a request to contact someone on Telegram is a meaningful combination in this domain specifically. Fine-tuning those attention layers with LoRA gives the model room to learn exactly that, to raise its attention score when it sees a payment method mentioned alongside off-platform contact instructions, because that pairing is a genuine fraud signal here, even if it wouldn't mean anything in a different context.

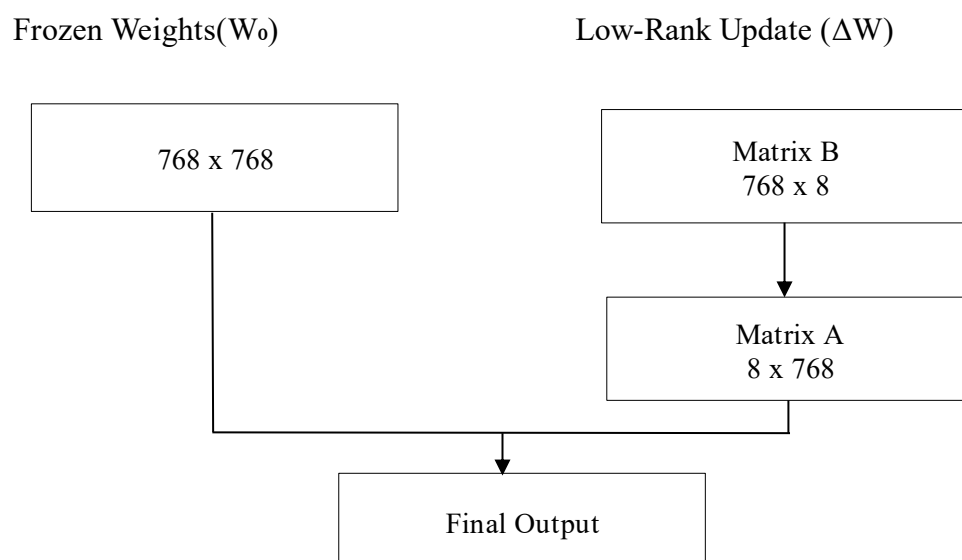


Figure 5.1: Low-Rank Adaptation (LoRA)

The end result is that the CLS embeddings coming out of the model would carry more domain-specific signal than they do now, which should give the downstream MLP classifier a stronger foundation to work from.

5.3.2 Multi-Modal Verification Frameworks and Security Fusion

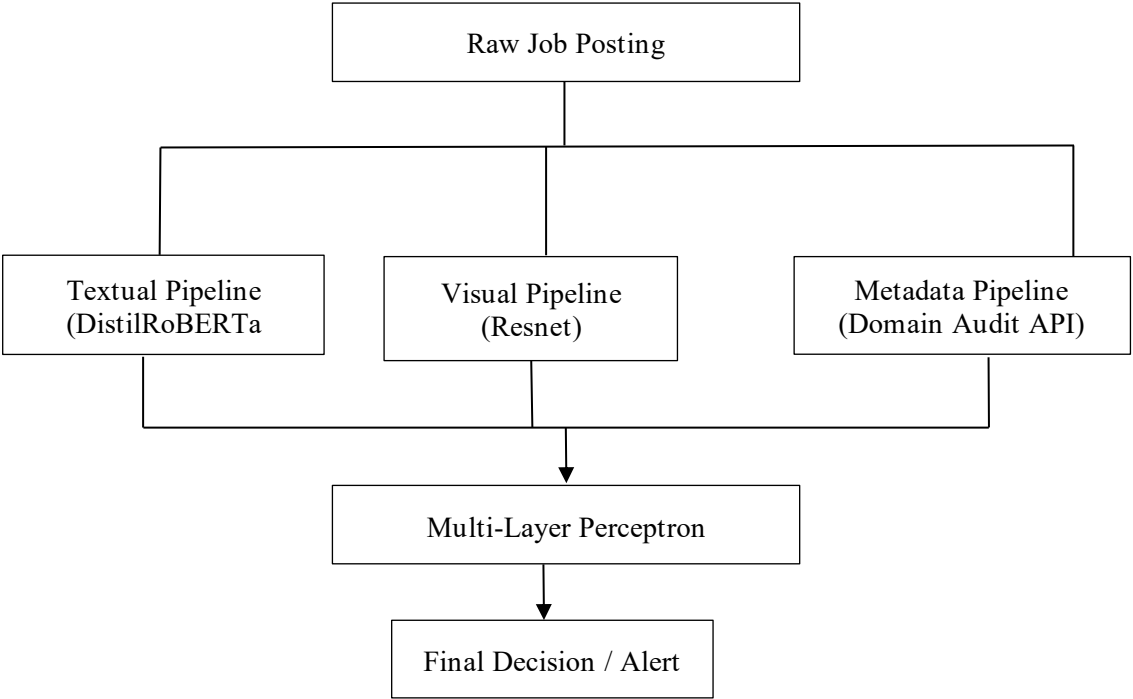


Figure 5.2: Multi-Modal Feature Fusion Framework

A lot of these scams eventually involve sending the applicant a PDF or Word document, something dressed up as an interview form or a background-check request. Scanning these for inconsistencies, checking whether the listed document author actually matches the company's officers, and screening for malicious links or embedded macros would add one more layer of protection, catching the phishing and malware angle that text analysis alone was never built to see.

5.3.3 Dynamic Adaptation, Continual Learning, and Concept Drift Mitigations

Fraud tactics don't hold still. Scammers keep shifting their language, their payment methods, the platforms they push candidates toward, precisely because that's what it takes to stay ahead of whatever filter is currently catching them. A model trained once

on a fixed dataset and left alone will inevitably suffer from concept drift, getting quietly worse over time as the world it was trained on stops matching the world it's actually operating in. The way to guard against that is a continuous learning loop, built around a few stages: Catching what slips through. When a candidate spots a scam listing that the automated filter missed, they can report it. A human checks it.

A security analyst reviews the report, and if it's confirmed as genuine fraud, it gets added to a rolling database rather than acted on immediately. The model gets updated, not rebuilt. Periodically, the classifier head is retrained on this growing, updated database. The tricky part of this loop is that neural networks have a habit of catastrophic forgetting, learning new patterns tends to come at the cost of quietly overwriting older ones. A model that gets very good at catching this year's scams can, in the process, lose its grip on last year's. Elastic Weight Consolidation is the safeguard here. The idea is to figure out which weights in the model actually mattered for its earlier knowledge, and make those specific weights harder to change during retraining. That importance is measured using the diagonal of the Fisher Information Matrix, which essentially captures how sensitive the model's loss is to nudging each individual parameter, if a small change to a weight would badly hurt performance on old data, that weight is treated as important and protected. Practically, this means adding a penalty term to the loss function during retraining:

$$\text{Loss_total} = \text{Loss_new} + \sum [(\lambda / 2) \times F_i \times (\theta_i - \theta_{\text{old},i})^2]$$

Here Loss_new is just the ordinary cross-entropy loss on the new scam examples. θ_i is what the parameter is being nudged toward now, and $\theta_{\text{old},i}$ is where it sat before this round of retraining. F_i is that Fisher information value, how much this particular weight mattered historically, and λ controls how strict the whole penalty is. The effect is fairly intuitive once you see it laid out: weights that were doing important work for detecting older fraud patterns get anchored in place, while weights with little historical importance are left free to shift and absorb the new patterns. The model ends up learning what's new without quietly forgetting what it already knew, which is really the whole point of treating this as an ongoing process rather than a one-time training run.

5.3.4. Distributed Edge Architectures, WebAssembly, and Client-Side Safety

Running this at real scale, watching millions of listings across multiple platforms, comes with a cost that isn't just about compute. A centralized server processing and storing every listing, every resume, all message thread candidates exchange with recruiters becomes both expensive to run and a genuinely attractive target for a data breach. The more sensitive data sitting in one place, the more there is to lose if that place gets compromised.

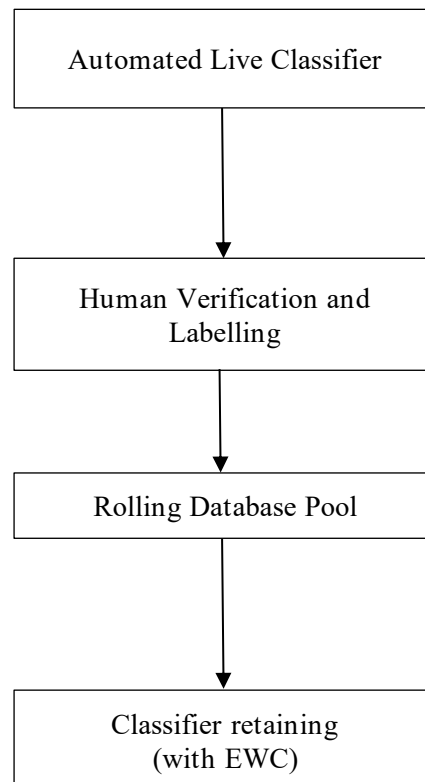


Figure 5.3: Continual Learning Feedback Loop with Elastic Weight Consolidation(EWC)

A distributed, client-side deployment sidesteps a lot of this, and it's a fairly natural next step given how lightweight the model already is: Shrinking the model down. Post-training quantization reduces the model's weight from 32-bit floating point down to 8-bit integers, which itself reduces the size from around 320MB down to roughly 80MB, with only minor hit to classification accuracy. That's a big enough reduction to matter for something meant to run on an ordinary device rather than a server.

Getting it into the browser. Once quantized, the model can be compiled into ONNX format and loaded directly into a browser extension via ONNX Runtime Web, no server round-trip required just to run inference. Doing the classification locally. As a candidate browses a listing, the extension tokenizes and evaluates the text right there in the browser. The privacy payoff here is the real point. A candidate's search history, resume content, cover letters, none of that ever needs to leave their own machine, because the classification is happening locally rather than being shipped off to a central server for processing. The server only re-enters the picture when the local model flags something with high confidence, and even then, only anonymized text features get sent along for moderation, not the candidate's personal browsing activity.

5.3.5 Ethical Governance, Fairness Auditing, and Regulatory Compliance

Screening tools like this one don't get to stay purely technical for long. Once they start touching people's employment opportunities, they land squarely in regulatory territory, and a few obligations come with that.

Fair play, algorithmically speaking. The EU AI Act puts systems that effect hiring decisions in the high-risk bucket. This means transparency and traceability aren't optional extra, they're base requirements, and bias has to be checked for actively, not just assumed a way. A toolkit like AI Fairness 360 would be a sensible addition here, run periodically to watch whether the model treats different job categories, remote roles, or smaller company profiles consistently, rather than trusting that it does simply because early testing didn't turn anything up.

Letting moderators actually see what's happening. A flag with no reasoning attached isn't much use to a human reviewer. What's needed is a dashboard where a moderator can look at a flagged post and see the SHAP breakdown behind it, which words pushed toward fraud, which pushed away from it. Something along these lines: A posting sitting at 87% fraud probability might show "Telegram contact" and "wire transfer" as the words driving that number up, while "equal opportunity employer" pulls slightly in the other direction. The moderator then either signs off on the flag or overrides it as a mistake.

That override path matters as much as the flag itself, it's what keeps a human genuinely in the loop instead of just rubber-stamping the model. Right to an explanation, GDPR-style. If a recruiter's post gets flagged, GDPR says they're entitled to know why. This one's more or less already covered by design, since SHAP is already producing word-level attributions as part of how the pipeline works. The same explanation a moderator uses to sanity-check the model doubles as the explanation that satisfies the legal requirement, no separate justification system needed on top.

5.3.6 Conclusion

This project set out to build something automated and explainable for catching online recruitment fraud, and it landed there. Splitting feature extraction from the actual classification decision turned out to be the right call, it keeps the accuracy where it needs to be while staying light enough to actually run on ordinary hardware rather than needing specialized infrastructure. A few things stood out clearly from the evaluation: Splitting the pipeline into two stages holds up in practice, not just in theory. A fixed pre-trained transformer feeding into a shallow classifier runs in within 100 milliseconds per listing on a standard CPU, fast enough for real deployment. Rebalancing the data is not optional, it's what makes this whole thing work. Without SMOTE, the model was catching barely a fifth of actual scams while still posting a deceptively high accuracy score.

With it, recall on fraud jumped to 87%, which is the difference between a system that looks good on paper and one that actually does its job. And explainability is not a nice-to-have bolted at the end, it's part what makes this trustworthy. Being able to point out to actual words, off-platform contact requests, payment demands, that drove a decision is what lets a human verify the model rather than just take its word for it. None of this is finished, of course. The system only reads English right now, and that's a real gap.

REFERENCES

- [1] G. Sravani Prasad and A. Emmanuel Raju, "Deep learning solutions for detecting fraud in online recruitment," *International Journal of Engineering Technology and Management Sciences*, vol. 9, no. 2, pp. 127-132.
- [2] K. S. Asif Mohiddin, Dr. Gopinathan, and P. Viswanatha Reddy, "Deep learning based detection of fraud in online recruitment," *International Journal of Advanced Research and Innovations*, vol. 15, no. 1, pp. 89-95.
- [3] Bathala Gireesh and Dr. P. Suribabu, "Online recruitment fraud (ORF) detection using deep learning approaches," *Journal of Engineering Sciences*, vol. 16, no. 3, pp. 501-510, 2025.
- [4] Gunasekaran K and Deepika A. D., "An automated deep learning approach to identify and prevent online recruitment fraud through job posting analysis," *International Journal of Multidisciplinary Research in Science, Engineering and Technology*, vol. 8, no. 8, pp. 12190-12196.
- [5] Dr. B. Praveen, "A deep learning framework for detecting fraudulent online job postings," *Anusandhanvallari*, vol. 2023, no. 1, pp. 170-177.
- [6] Ms. S. Sai Jaswanthi, Dr. D. Bright Anand, Dr. R. Karunia KrishnaPriya, Mr. Pandreti Praveen, and Dr. T. Senthil, "Detecting online recruitment fraud through deep learning using hybrid neural networks."
- [7] Sheeza Habeeba Khan and Dr. I. Samuel Peter James, "Deep learning-based methods for online recruitment fraud detection," *International Journal of Engineering Science and Advanced Technology (IJESAT)*, vol. 25, no. 9, pp. 87-93.
- [8] Dr. P. Vara Prasad, N. Lakshmi Sravya, M. Sree Kavya, V. Kavitha, and Shaik Sanafrin, "Online recruitment fraud detection using deep learning," *International Journal of Progressive Research in Engineering Management and Science (IJPREAMS)*, vol. 5, no. 4, pp. 1697-1703.
- [9] Sunkary Gnana Dipikhaa, Kothakonda Akruthipriya, Dherangulu Pavithra, and G.

- Shekar, "Online recruitment fraud detection using deep learning," International Journal of Advanced Research in Education and Technology (IJARETY), vol. 12, no. 3, May-June 2025.
- [10] P. Durga Pavan Kalyan and Dr. M. Sumender Roy, "Online recruitment fraud detection using machine learning approach," International Journal for Advanced Research in Science and Technology (IJARST), ISSN: 2457-0362.
- [11] Shwani Dutta and Prof. Samir Kumar Bandopadhyay, "Fake job recruitment detection using machine learning approach," International Journal of Engineering Trends and Technology (IJETT), vol. 68, no. 4, pp. 48-53.
- [12] Natasha Akram, Rabia Irfan, Ahmad Saml Al-Shamayaleh, Adila Kousar, Abdul Qaddos, Muhammad Imran, and Adnan Akhunzada, "Online recruitment fraud (ORF) detection using deep learning approaches," IEEE Access.
- [13] Suggu Thirupathi and M. Poojitha, "Online recruitment fraud (ORF) detection using deep learning approaches," International Journal of Research Publication and Reviews, vol. 6, no. 5.
- [14] K. Sravanthi, Y. Bhavani, A. Nikitha, and Ch. Murali Krishna, "Online recruitment fraud detection (ORF) using deep learning approaches," International Journal of Scientific Research in Engineering and Management (IJSREM), vol. 9, no. 6, June 2025.
- [15] Raza et al., "Online recruitment fraud detection using NLP and deep learning techniques," International Journal of Scientific Research in Engineering and Management, 2025.